



机器学习-吴恩达

🕒 Created	@2025年4月16日 21:44
📁 Class	AI

前言：

大二上选择的机器学习专业课需要上传机器学习的笔记，对应教材是吴恩达的机器学习，本来是打算直接用大一学习的笔记的，但是发现大一学习的时候看的竟然是2018版的吴恩达课程，当然老师给的也是2018版的。

于是就顺便看下2022版的课程 顺便做个笔记 看看对比。

实际上记下来之后感觉主要变化就是迎合了现在深度学习主流的时代，把一些机器学习中的算法（比如支持向量机）直接淘汰不讲。深度学习部分还是用的tensorflow，建议直接听李沐的吧。强化学习部分深入了一点 但是还是不是特别深入，还是建议单独去学吴恩达强化学习部分的课程。

课程为最新的吴恩达课程，主要为笔记部分，参考了2018版的诸多笔记，代码部分见jupyter。

已上传至github 个人博客就不上传了，反正也没人看，传起来怪费事。

2025.5.30

单变量线性回归

1.引言

1.1 欢迎

第一个视频主要讲了什么是机器学习，机器学习能做些什么事情，简单介绍下机器学习。

1.2 机器学习是什么？

举了一个机器学习跳棋的例子来介绍机器学习的作用，并介绍了课程的大纲。

2. 监督学习与无监督学习

2.1 监督学习

回归问题是一种常见的监督学习：

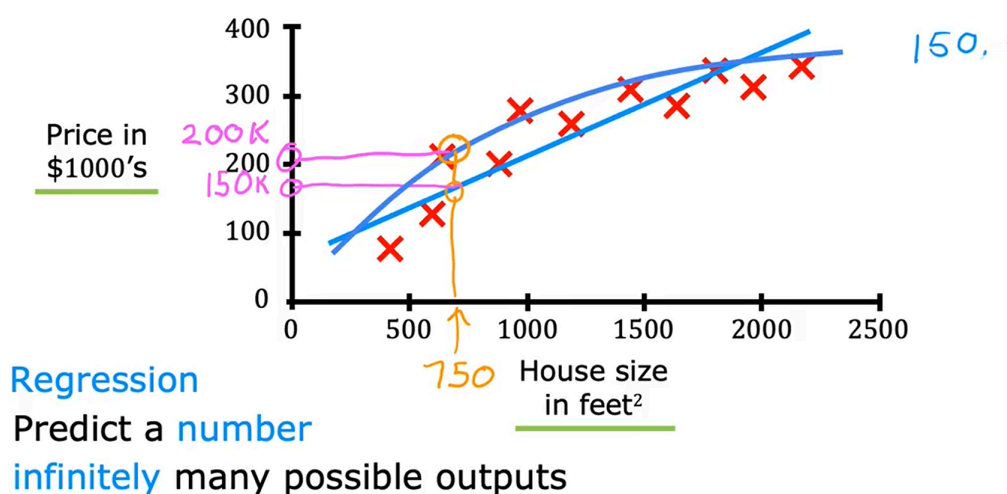


Learns from being given "right answers"

Input (X)	Output (Y)	Application
email	spam? (0/1)	spam filtering
audio	text transcripts	speech recognition
English	Spanish	machine translation
ad, user info	click? (0/1)	online advertising
image, radar info	position of other cars	self-driving car
image of phone	defect? (0/1)	visual inspection

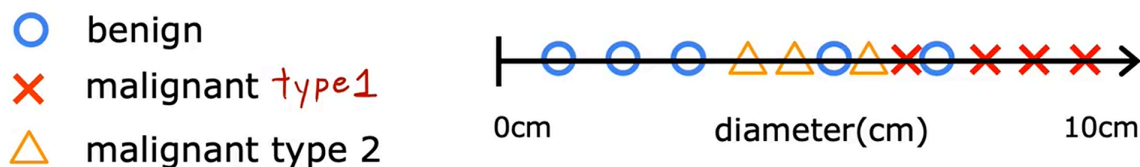
这些预测性的问题叫做回归问题：

Regression: Housing price prediction



还有一种叫做分类问题，也是监督学习常见的问题：

Classification: Breast cancer detection



class category

Classification

predict *categories* cat dog benign malignant 0, 1, 2

small number of possible outputs

很明显，分类问题的y是类别，而回归问题的y是数字。

总结：

Supervised learning

Learns from being given "*right answers*"

Regression

Predict a *number*

infinitely many possible outputs

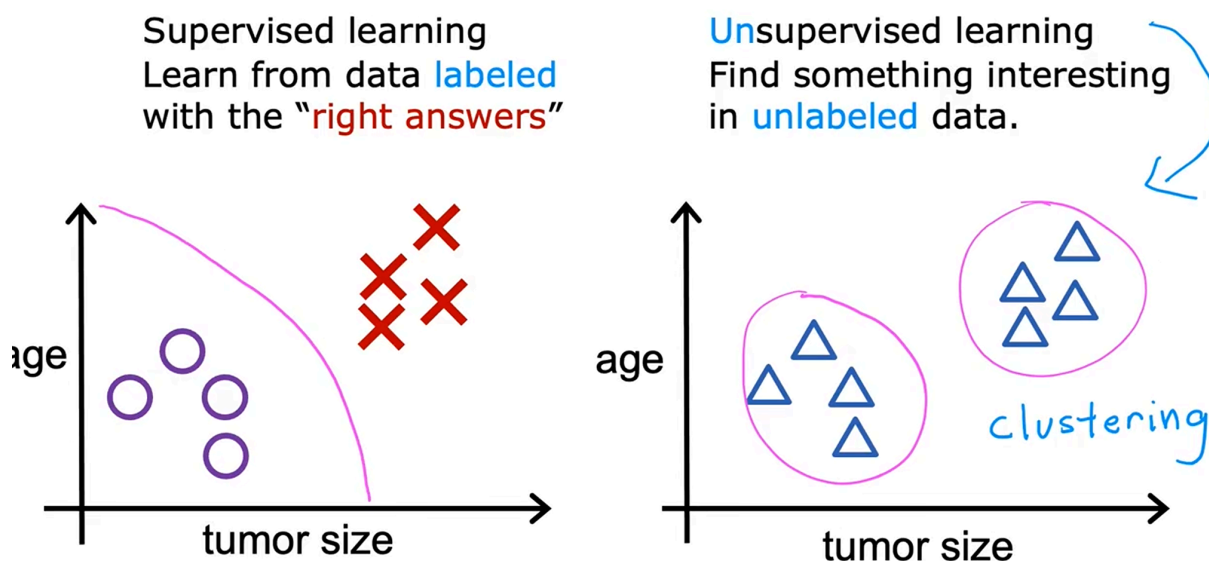
Classification

predict *categories*

small number of possible outputs

注意：监督学习是通过我们给入的正确答案（实际是相关标签）进行学习的，这是与后面无监督学习的本质区别。

2.2 无监督学习

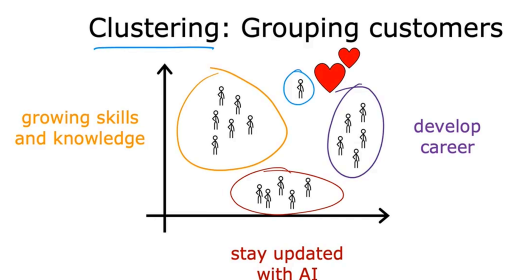
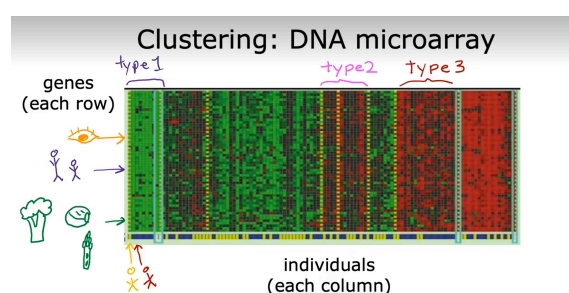


无监督学习不依赖与用户提供的标签。

这里的“无监督”指的是我们**不提供预先定义的、正确的输出或类别**来指导算法的学习过程。算法仍然需要遵循其内部的数学规则和优化目标（例如，最小化簇内距离，最大化簇间距离），但它是在没有“标准答案”的情况下，自主地在数据中寻找模式或结构。

其核心任务是从未标记的数据（即没有预先分配标签或类别的数据）中学习和推断模式、结构或关系。算法自主地探索数据，试图发现数据内在的组织规律。

聚类问题就是一种典型的无监督学习问题



上面展示了几个聚类问题的经典例子。

第一个是谷歌新闻，算法可以自动的把相关新闻进行聚类然后推送，很明显我们是没有告诉算法第一个新闻是关于panda和zoo的，并且把所有新闻都提取标签是不现实的，算法是自动进行聚类分析的。

第二个是DNA分类，通过给出大量的DNA序列数据然后进行特性划分，很显然我们没有告诉每一个type应该是什么样的，是算法自己分析DNA序列数据进行的分类

第三个是网站的喜好分类，通过每个人的浏览数据来定义每个人的标签，然后进行聚类。明显，每个人的标签和聚类的标签都是算法生成的，我们没有干预。

异常分析和降维也是常见的无监督学习

无监督异常检测是指在不依赖任何预先标记的训练数据（即没有标签指明哪些是正常或异常）的情况下，识别出数据集中与大部分数据显著不同的数据点或模式的过程。算法通常基于数据的内在结构、分布密度、距离关系或其他统计特性来学习“正常”模式，并将偏离这些模式的数据点识别为异常。

无监督异常检测算法通常假设：

1. 正常样本在数据空间中占据密集的区域，彼此相似。
2. 异常样本数量稀少，并且在特征上与正常样本有显著差异，可能位于数据空间的稀疏区域，或者远离密集的正常数据簇。

降维是指将高维空间中的数据映射到低维空间的过程，目标是在减少数据复杂性、存储需求和计算成本的同时，最大限度地保留与任务相关的数据结构、方差或信息。这个过程可以是有损的（丢失部分信息）或无损的（理论上可完全恢复，但通常指保留关键信息）。

3.线性回归

3.1 训练集以及相关术语

Terminology	
Training set:	Data used to train the model
x	y
size in feet ²	price in \$1000's
(1) 2104	400
(2) 1416	232
(3) 1534	315
(4) 852	178
...	...
(47) 3210	870
$m = 47$	
$x^{(4)} = 2104$	$y^{(4)} = 400$
$(x^{(4)}, y^{(4)}) = (2104, 400)$	
$x^{(2)} = 1416$	$x^{(2)} \neq x^2$ not exponent

Notation:
 x = "input" variable
feature
 y = "output" variable
"target" variable
 m = number of training examples
 (x, y) = single training example
 $(x^{(i)}, y^{(i)})$
 $(x^{(i)}, y^{(i)}) = i^{\text{th}}$ training example
index (1st, 2nd, 3rd ...)

m 代表训练集中实例的数量

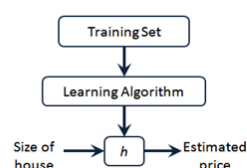
x 代表特征/输入变量

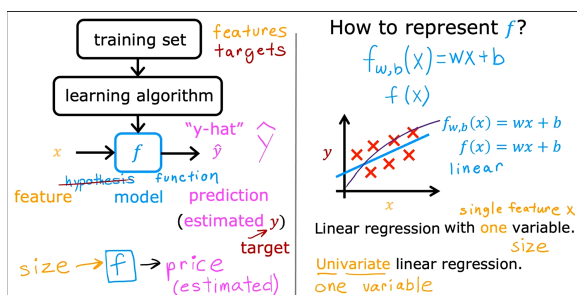
y 代表目标变量/输出变量

(x, y) 代表训练集中的实例

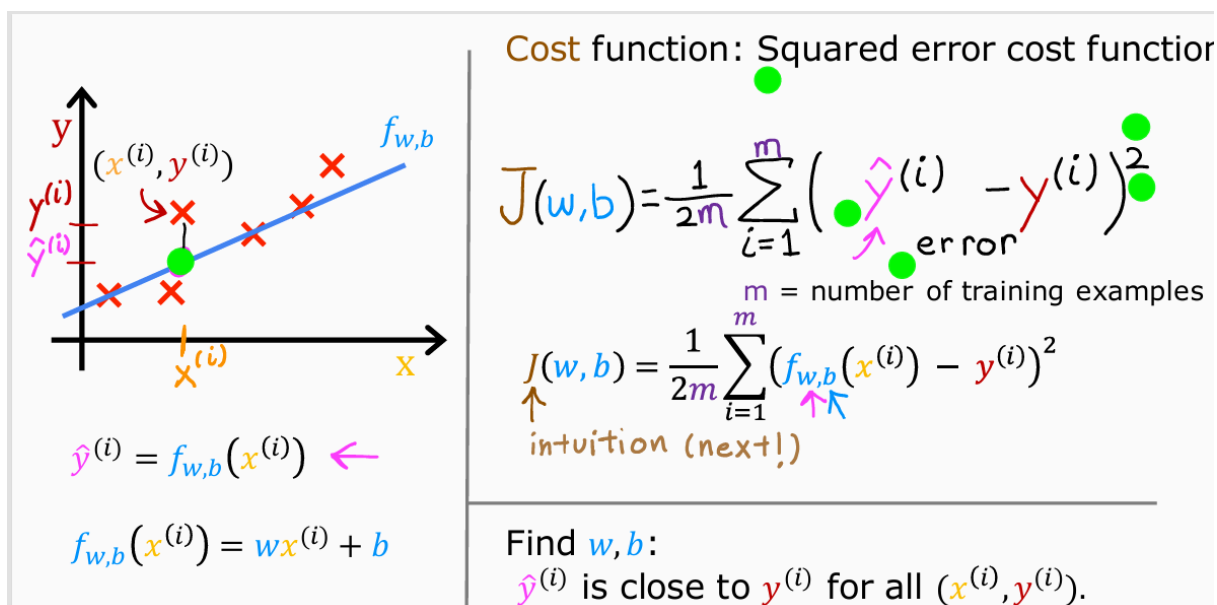
$(x^{(i)}, y^{(i)})$ 代表第 i 个观察实例

h 代表学习算法的解决方案或函数也称为假设 (hypothesis)





3.2 代价函数 (Cost Function)

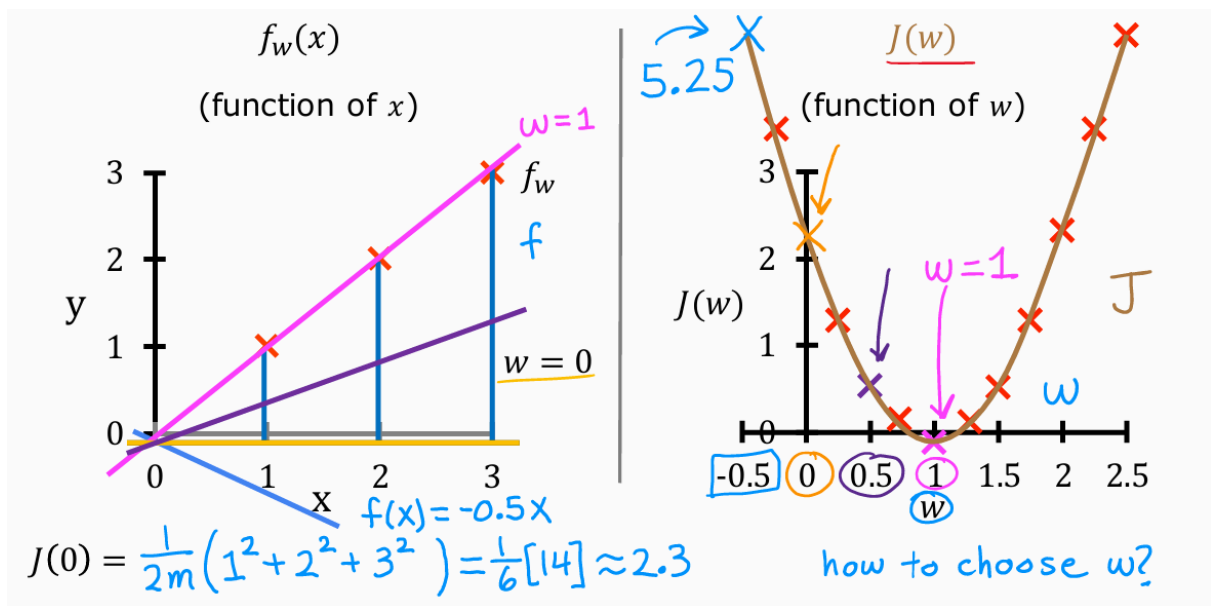


上图展示了最经典的代价函数—均方误差

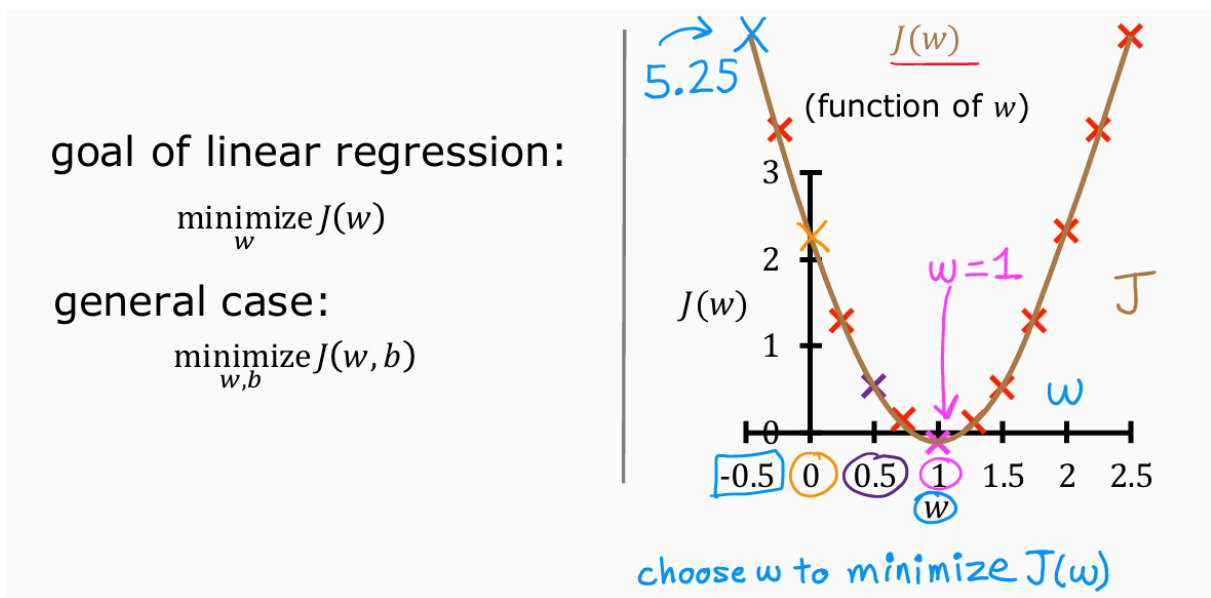
这里有的时候会 $\div 2m$ ，由于我们在代价函数定义中预先加入了 $1/2$ ，它正好与求导时从平方项产生的因子 2 相互抵消了。

3.3 代价函数的直观理解

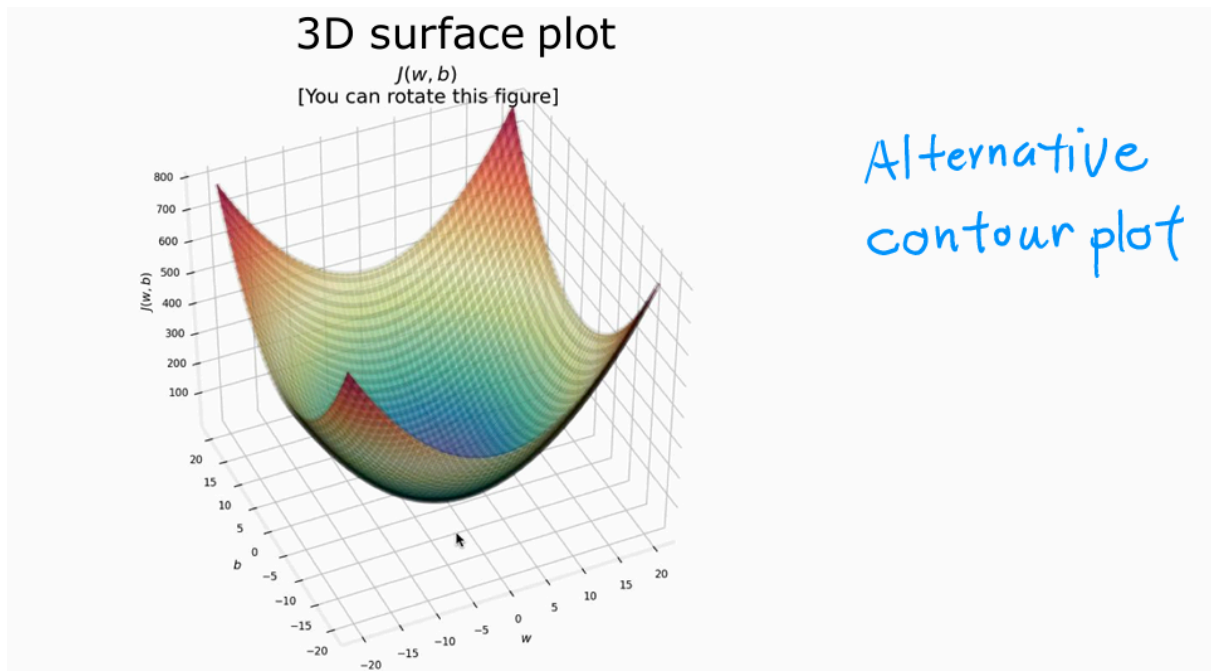
先假设下 b 为 0 ，只考虑 w



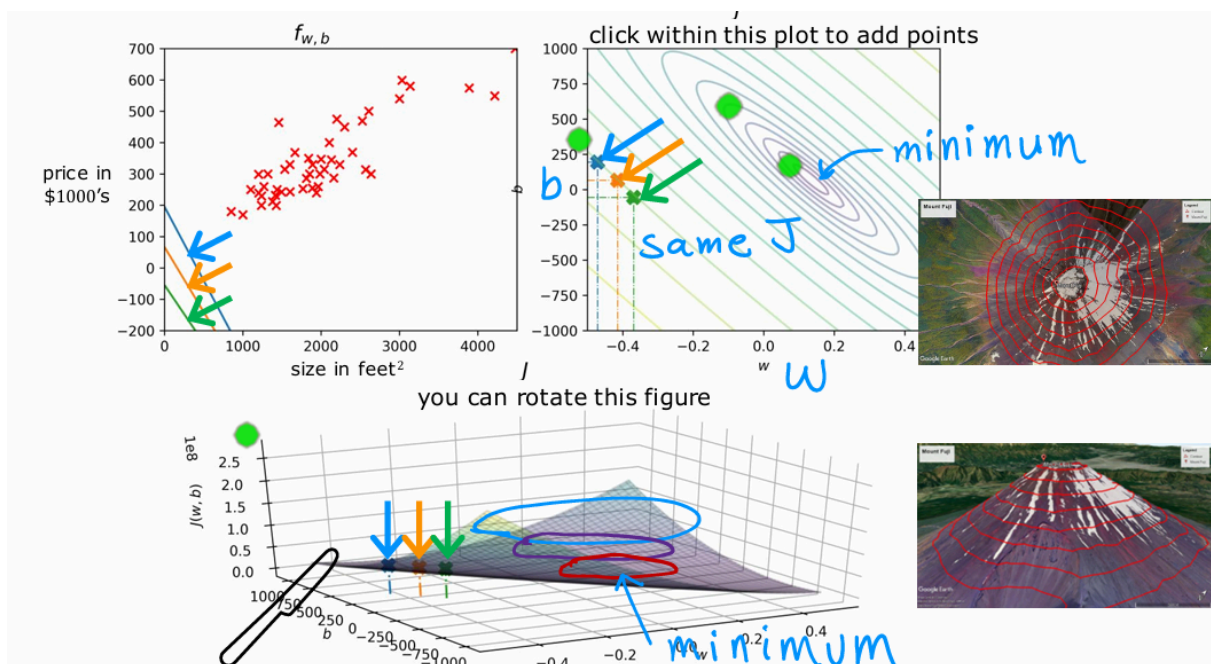
在上面的图片中，通过改变左图不同的 w 值，对应的 $J(w)$ 函数值如右图所示
 可以发现是一个二次函数，可以求最小值，最小值对应的 w 就是损失最小的 w ，也就是我预测函数的 w 。



如果加入了 b ，那么右图就会变成 $J(w, b)$ 与 w, b 的关系，就是一个三维图



做出三维图的等高图，明显等高图中每一个椭圆都是在不同 w, b 下的相同 $J(w, b)$ 的集合，而且最小椭圆的中心就是最小值

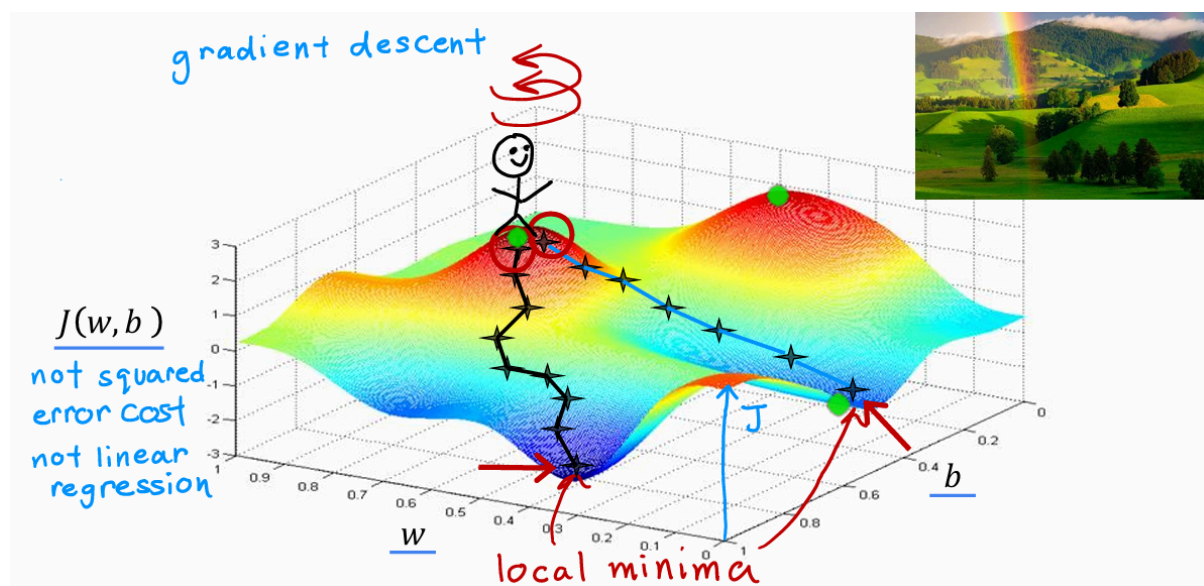


接下来就是要用梯度下降算法来找到这个最小点

3.4 梯度下降算法

根据高数所学知识可以理解梯度下降算法的原理

使用均方误差作为代价函数永远是凸函数，只有一个最小值，但是如果使用的是其他代价函数就会出现下面的情况：



出现多个最小点（低谷），运用梯度下降算法容易走到不同的最小点（低谷）上，这些最小点叫做局部最小值，并且每个局部最小值都不能用梯度下降算法到另一个最小值上。

这是梯度下降算法本身特性，想要解决这个问题没有一个确定解，只能找到最优解决办法。

1. 使用动量 (Momentum):

- **原理:** 模拟物理世界中的惯性。参数更新不仅仅取决于当前的梯度，还取决于之前的更新方向。就像一个滚下山的球，即使到了一个平缓的区域或者一个小坑（浅的局部最小值），由于惯性，它还会继续滚动一段距离，有可能“冲”出这个小坑，继续寻找更低的地方。
- **效果:** 有助于加速收敛，并且能够越过一些浅的局部最小值或平坦区域（鞍点）。

2. 使用随机梯度下降 (Stochastic Gradient Descent, SGD) 或小批量梯度下降 (Mini-batch Gradient Descent):

- **原理:** 标准梯度下降（也叫批量梯度下降 Batch GD）在每次更新时都使用全部训练数据计算梯度，方向是精确的。而 SGD 每次只用一个样本计算梯度，Mini-batch GD 用一小批样本。这使得每次计算的梯度带有噪声（因为基于部分数据，梯度方向会有随机波动）。

- **效果:** 这种噪声有时反而是一件好事！它可能使得参数更新方向偶尔偏离“最陡峭”的局部下降方向，这种随机的“抖动”**有可能将参数“踢”出局部最小值的陷阱**，使其有机会探索参数空间的其他区域。Mini-batch GD 在计算效率和噪声利用之间取得了较好的平衡。

3. 自适应学习率算法 (Adaptive Learning Rate Algorithms):

- **代表:** AdaGrad, RMSprop, Adam (Adaptive Moment Estimation) 等。
- **原理:** 这些算法不再使用固定的学习率 α ，而是为**每个参数**动态地调整学习率。通常，对于梯度较大的参数，学习率会减小；对于梯度较小的参数，学习率会增大（或者说，它们会根据历史梯度信息调整更新步长）。
- **效果:** 能够更有效地在复杂的代价函数曲面上导航。例如，Adam 结合了动量和 RMSprop 的优点，是目前非常流行且效果通常很好的优化器，它在处理局部最小值和鞍点（Saddle Point，梯度也为零但不是最小值点）方面表现更佳。

4. 随机重启 (Random Restarts) / 多次初始化 (Multiple Initializations):

- **原理:** 多次**随机地初始化**模型参数 θ ，然后对每一次初始化都运行梯度下降算法，直到收敛。最后，比较所有运行结果，选择那个使得代价函数值**最低**的解作为最终结果。
- **效果:** 通过从不同的起点开始搜索，大大增加了至少有一次能够找到全局最小值或一个非常接近全局最小值的解的概率。这是最简单直接但也比较有效的方法之一，缺点是计算成本会增加。

5. 学习率退火/调度 (Learning Rate Annealing/Scheduling):

- **原理:** 在训练初期使用较大的学习率，让参数能够快速移动，探索更广阔的参数空间（有助于跳出早期可能遇到的局部最小值）。随着训练的进行，逐渐减小学习率，使得参数能够更稳定地收敛到找到的最小值区域内。
 - **效果:** 平衡了探索 (Exploration) 和利用 (Exploitation)。
-

Gradient descent algorithm

Repeat until convergence

$$\begin{cases} \underline{w} = w - \alpha \frac{d}{dw} J(w, b) \\ \underline{b} = b - \alpha \frac{d}{db} J(w, b) \end{cases}$$

Learning rate
Derivative

Simultaneously
update w and b

Assignment

$$a = c$$

$$a = a + 1$$

Code

Truth assertion

$$a = c$$

$$a = a + 1$$

Math

$$a == c$$

Correct: Simultaneous update

$$tmp_w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

$$tmp_b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

$$w = tmp_w$$

$$b = tmp_b$$

Incorrect

$$tmp_w = w - \alpha \frac{\partial}{\partial w} J(w, b)$$

$$w = tmp_w$$

$$tmp_b = b - \alpha \frac{\partial}{\partial b} J(w, b)$$

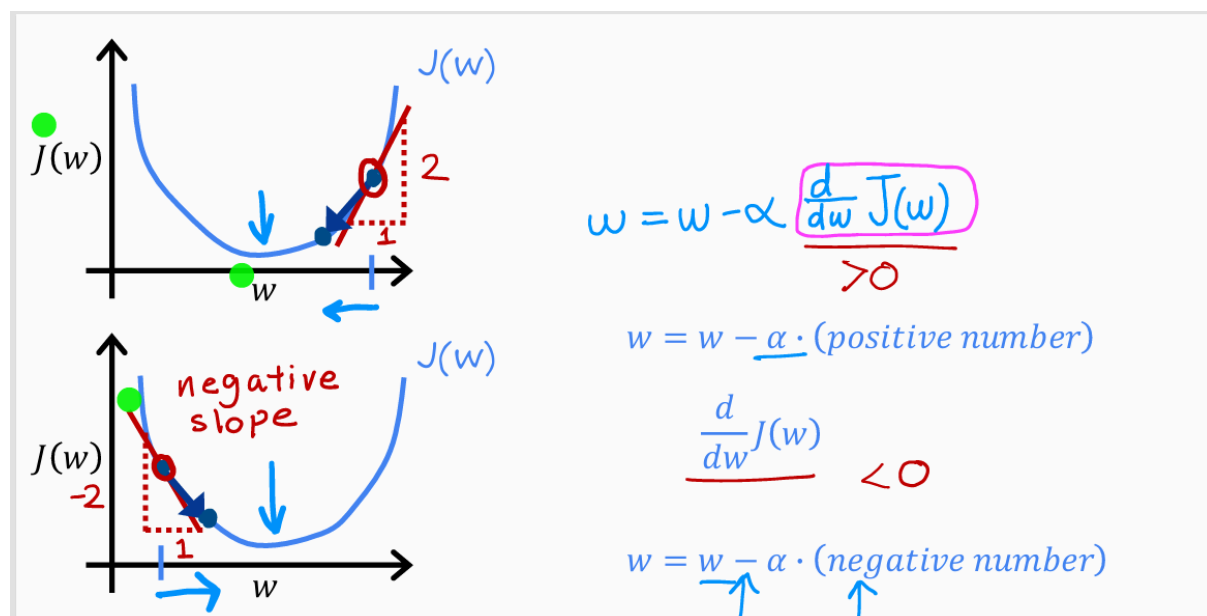
$$b = tmp_b$$

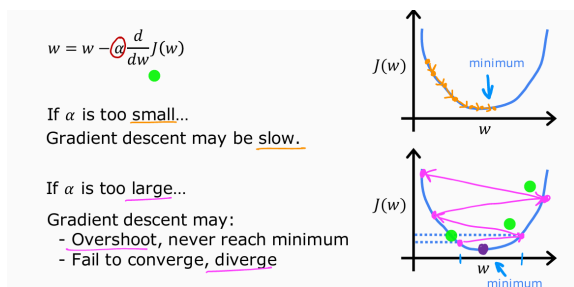
注意下面的对比，梯度下降算法需要同步更新w,b。

右面错误的代码进行了异步更新，可以看到后面的b取决于更新后的w，这通常是不对的。

3.4 学习率

和高数学的一样，梯度下降算法依靠导数的变化来找到最低点





学习率的改变会直接影响梯度下降的速度，如果太小就会下降的特别慢。

如果太大则会在接近最小值后又超过了最小值，然后反复地超过最小值，一直在震荡，导致永远达不到最小值。

3.5 一元线性回归

结合上面所有的内容，就可以完成对一元（线性）函数的回归预测

Linear regression model

$$f_{w,b}(x) = wx + b$$

Cost function

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

Gradient descent algorithm

repeat until convergence {

$w = w - \alpha \frac{\partial}{\partial w} J(w, b)$
→
 $\frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$

$b = b - \alpha \frac{\partial}{\partial b} J(w, b)$
→
 $\frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$

}

next slide is optional!

推到过程如下，可以看到之前为什么要 $\div 2m$

(Optional)

$$\frac{\partial}{\partial w} J(w, b) = \frac{\partial}{\partial w} \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2 = \frac{\partial}{\partial w} \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)})^2$$

$$= \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)}) \cdot 2x^{(i)} = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})x^{(i)}$$

$$\frac{\partial}{\partial b} J(w, b) = \frac{\partial}{\partial b} \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2 = \frac{\partial}{\partial b} \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)})^2$$

$$= \frac{1}{2m} \sum_{i=1}^m (wx^{(i)} + b - y^{(i)}) \cdot 2 = \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$$

no $x^{(i)}$

更准确的，这种梯度下降算法叫做批次梯度下降算法

原因是在更新每一次 w , b 时，我们都对所有的数据进行了求和，我们需要考虑所有这一"批"训练样本。

多元回归

1. 多元线性回归

1.1 多维特征

Multiple features (variables)

Size in feet ²	Number of bedrooms	Number of floors	Age of home in years	Price (\$) in \$1000's
x_1	x_2	x_3	x_4	
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

$j = 1 \dots 4$
 $n = 4$

$i = 2$

$x_j = j^{th}$ feature

n = number of features

$x^{(i)}$ = features of i^{th} training example

$x_j^{(i)}$ = value of feature j in i^{th} training example

$\vec{x}^{(2)} = [1416 \ 3 \ 2 \ 40]$

$x_3^{(2)} = 2$

n 代表特征的数量

$x^{(i)}$ 代表第 i 个训练实例，是特征矩阵中的第 i 行，是一个向量 (vector)。

比方说，上图的

$$x^{(2)} = \begin{bmatrix} 1416 \\ 3 \\ 2 \\ 40 \end{bmatrix}$$

$x_j^{(i)}$ 代表特征矩阵中第 i 行的第 j 个特征，也就是第 i 个训练实例的第 j 个特征。

如上图的 $x_2^{(2)} = 3, x_3^{(2)} = 2$ 。

支持多变量的假设 h 表示为: $h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$ 。

1.2 向量化

主要解释为什么要使用numpy数组来进行运算，为什么要把数据变成向量和矩阵来计算。

Without vectorization $n = 100,000$	Vectorization
$f_{\vec{w},b}(\vec{x}) = w_1x_1 + w_2x_2 + w_3x_3 + b$	$f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$
$f = w[0] * x[0] +$ $w[1] * x[1] +$ $w[2] * x[2] + b$	$f = \text{np.dot}(w, x) + b$
	

Without vectorization	Vectorization
for j in range(0,16): f = f + w[j] * x[j]	np.dot(w,x)
t_0 f + w[0] * x[0]	t_0 w[0] w[1] ... w[15]
t_1 f + w[1] * x[1]	t_1 x[0] x[1] ... x[15]
...	...
t_{15} f + w[15] * x[15]	t_{15} w[0]*x[0] + w[1]*x[1] + ... + w[15]*x[15]
	efficient $\rightarrow$ scale to large datasets

上图可以看出向量化后的数据是并行运算的，效率更高

1.3 多元线性回归的梯度下降

把一元的参数矢量化后就是多元线性回归

Gradient descent	
One feature	n features ($n \geq 2$)
repeat {	repeat {
$\underline{w} = w - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)}) x^{(i)}$	$\underline{w}_1 = w_1 - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_1^{(i)}$
$\vdots$	$\vdots$
$\underline{b} = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})$	$\underline{w}_n = w_n - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_n^{(i)}$
simultaneously update w, b	simultaneously update w_j (for $j = 1, \dots, n$) and b
}	}

下面又介绍了一种特殊的求最优解的方法，但是只适用于线性回归。

$$\theta = (X^T X)^{-1} X^T y$$

An alternative to gradient descent

→ Normal equation

- Only for linear regression
- Solve for w, b without iterations

Disadvantages

- Doesn't generalize to other learning algorithms.
- Slow when number of features is large ($> 10,000$)

What you need to know

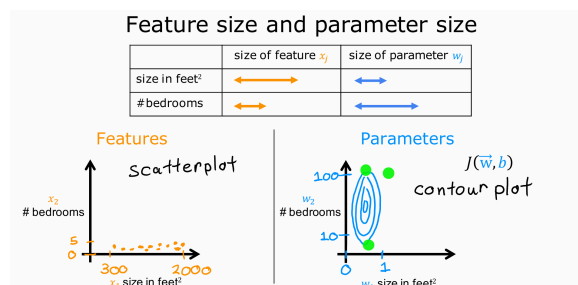
- Normal equation method may be used in machine learning libraries that implement linear regression.
- Gradient descent is the recommended method for finding parameters w, b

注意的是，上面的公式在计算线性回归的时候比梯度下降算法快，很多的许多机器学习库（如 scikit-learn）在特征维度较小的时候，会内部采用正规方程来快速训练线性回归模型。

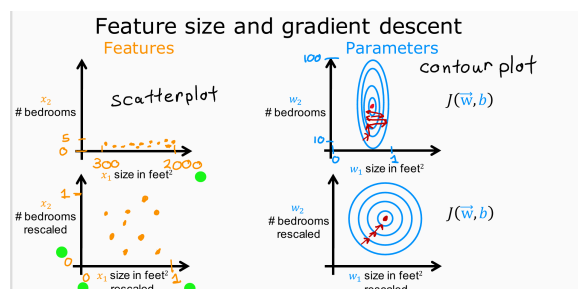
实际上两种算法都可以，而梯度下降算法更普遍，所以还是常用梯度下降算法。

2.特征缩放

2.1 为什么要特征缩放



从图中可以看出不同的参数应该要设置不同的特征量，小参数设置大的特征量，大参数设置小的特征量



在这张图中，借助等高图可以直观地发现缩放后的特征的等高图更接近于圆形，有利于梯度下降算法的进行，而缩放前的是椭圆形，每一个

- **圆形等高线——步伐一致、直取最优**

- 当等高线是圆形时，梯度 ∇J 始终指向圆心（最优解方向），而且在各个坐标轴方向上的曲率相同。
- 使用固定学习率 α 时，每一步更新会“直线”地收敛到最优解，不会来回摆动，收敛速度最快。

- **椭圆等高线——曲率不一、来回摆动**

- 椭圆的长轴方向对应的大特征值（陡峭方向），短轴方向对应小特征值（平坦方向）。
- 梯度沿着“陡峭方向”很大，在“平坦方向”很小。固定步长下：
 - 在陡峭方向会 *overshoot*（步子太大），
 - 在平坦方向会 *undershoot*（步子太小）。
- 因此更新路径会在两者之间“来回振荡”，既无法一步到位，也无法均匀前进，导致收敛慢甚至震荡不收敛。

2.2 特征缩放方法

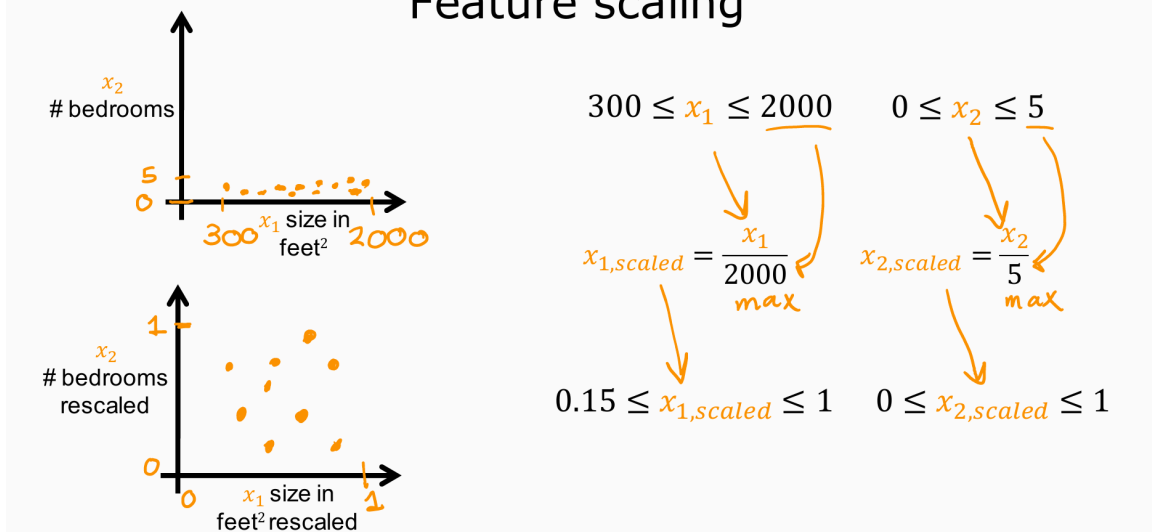
1. 最小-最大归一化（Min-Max Scaling）

- **公式：**

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

- **效果：**将所有特征线性映射到 $[0,1]$ $[0, 1]$ （也可映射到 $[a,b]$ $[a, b]$ ）。
- **优点：**保留原始数据的分布形状；区间固定。
- **缺点：**对异常值（outliers）非常敏感，最小值或最大值若有离群点，会拉伸大部分样本到很窄区间。

Feature scaling



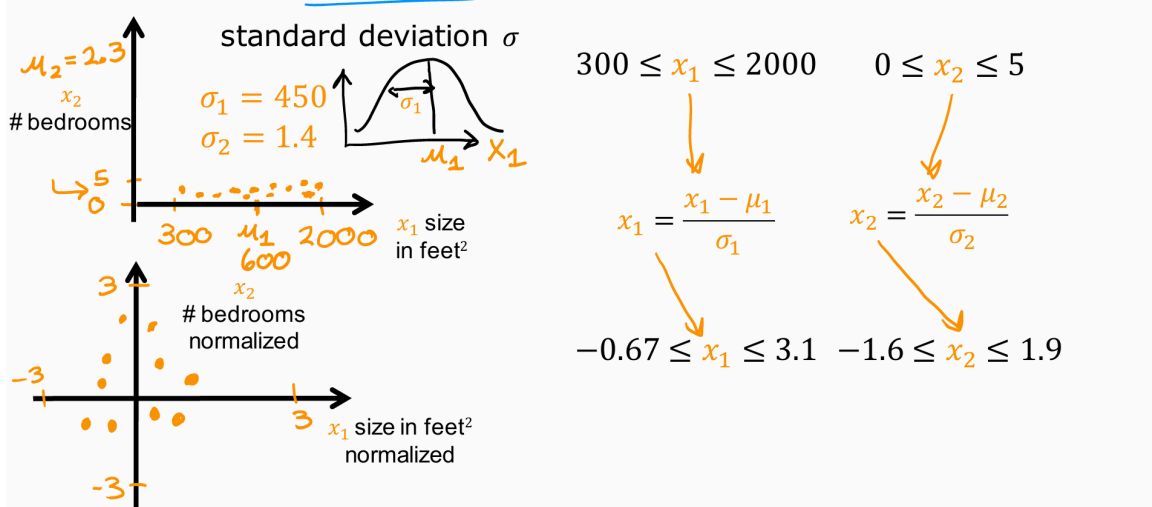
2. 标准化 (Z-Score 标准化)

- 公式:

$$x' = \frac{x - \mu}{\sigma}$$

- 效果: 输出特征具有零均值、单位标准差。
- 优点: 对异常值的影响相对较小; 适用于绝大多数需要假设数据服从正态分布的算法 (如线性回归、逻辑回归、SVM)。
- 缺点: 会改变原始数据分布形状 (如果原始分布偏态严重, 标准化后仍偏态)。

Z-score normalization

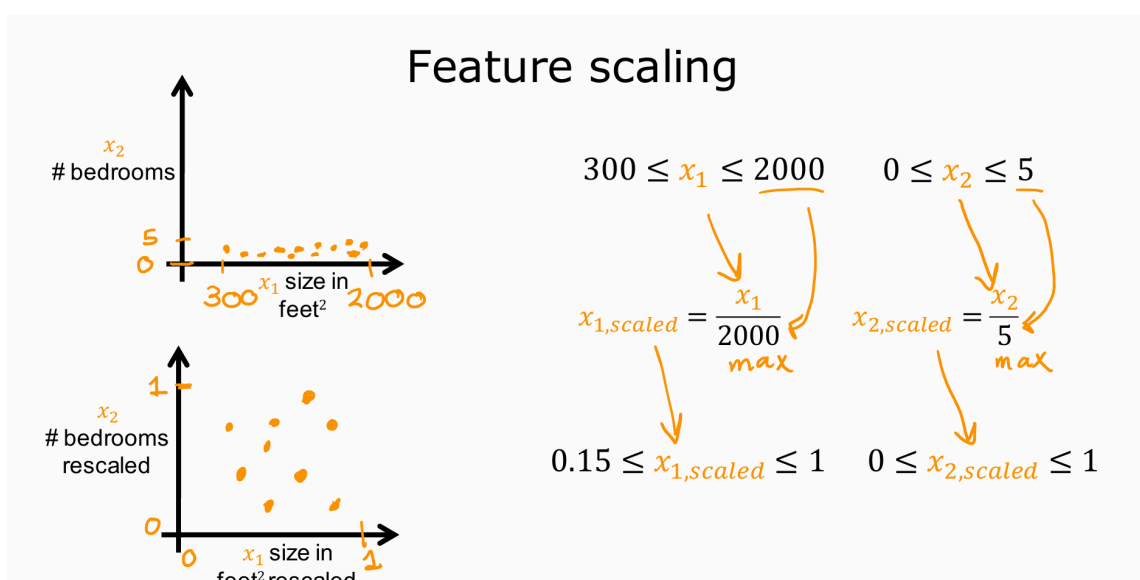


3. 最大绝对值缩放 (MaxAbs Scaling)

- 公式:

$$x' = \frac{x}{\max(|x|)}$$

- 效果: 将特征映射到 $[-1,1]$ 。
- 优点: 对稀疏数据友好 (不会将稀疏矩阵中的零值变为非零); 保留了数据的稀疏特性。
- 缺点: 对异常值仍然敏感, 但对零点保持绝对。



4. 鲁棒缩放 (Robust Scaling)

- 公式:

$$x' = \frac{x - Q_2}{Q_3 - Q_1}$$

其中 Q_1 、 Q_2 、 Q_3 分别是第 1、2、3 四分位数。

- 效果: 基于中位数和四分位距 (IQR) 进行缩放, 输出中心在 0, 尺度与 IQR 对齐。
- 优点: 对异常值极其不敏感, 特别适合存在离群点的数据。
- 缺点: 如果数据分布非常不对称, 中位数与 IQR 也可能偏离重要分位。

5. 单位向量归一化 (Normalization 或 L1/L2 范数缩放)

- **公式 (L2 归一化):**

$$x' = \frac{x}{\|x\|_2} = \frac{x}{\sqrt{\sum_i x_i^2}}$$

- **效果:** 将样本缩放为单位范数 (L1 或 L2)，常用于文本特征 (TF-IDF 向量) 或相似度计算。
- **优点:** 适合需要根据向量角度或相似度计算的场景；与余弦相似度配合良好。
- **缺点:** 只关注向量方向，忽略了特征原始尺度及分布信息。

6. 小数定标 (Decimal Scaling)

- **公式:**

$$x' = \frac{x}{10^j}, \quad j = \lceil \log_{10}(\max |x|) \rceil$$

- **效果:** 通过移动小数点位置将数值映射到 $[-1,1]$ 范围内。
- **优点:** 计算简单直观。
- **缺点:** 缩放粒度较粗，不如其它方法灵活。

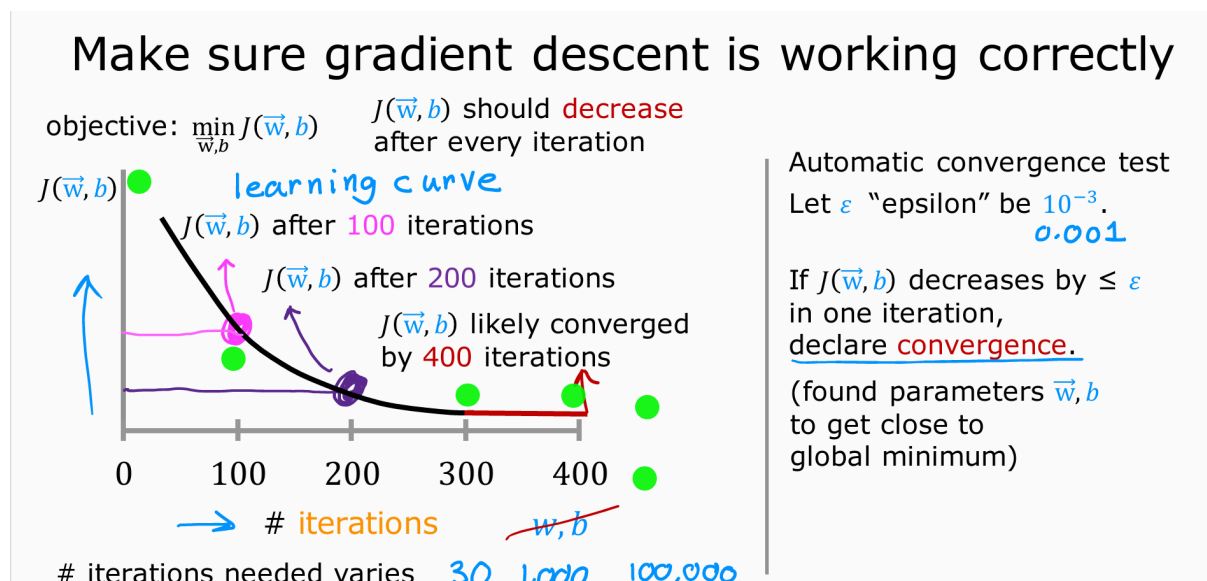
7. 分位数变换 (Quantile Transformer)

- **原理:** 将数据映射到均匀分布 (或正态分布)，通过分位数函数 (CDF) 与反 CDF。
- **效果:** 可以使任意分布变得接近指定分布 (如均匀或正态)，对于强偏态数据很有效。
- **优点:** 能有效消除偏态，提高某些算法效果。
- **缺点:** 会严重扭曲原始数据分布，不易解释。

选用建议

- 数据无明显离群点且有固定区间需求 → Min-Max Scaling。
- 数据近似正态分布或部分离群点 → Z-Score 标准化。
- 稀疏数据 (如文本向量)、保留零值 → MaxAbs Scaling。
- 存在较多离群点 → Robust Scaling。
- 关注向量方向、相似度计算 → L1/L2 归一化。
- 需要映射到特定分布 → 分位数变换。

2.3 判断梯度下降是否收敛



左侧方法：学习曲线（Learning Curve）可视化检查

- 如果曲线在某个迭代区间（比如 200→400 次）几乎保持平坦，说明损失不再显著下降，训练很可能已接近全局最优或某个良好局部最优。
- 通过“目测”什么时候曲线开始变平、下降幅度趋近于 0，就可以决定何时停止迭代。
- 优点：
 - 直观，可以同时发现“学习率过大导致抖动”“过小导致收敛太慢”等问题；
 - 对学习率、批大小、正则化等超参的影响一目了然。
- 缺点：
 - 需要实时绘图或记录，操作上相对繁琐；
 - “何谓平缓”多少是主观判断，难以自动化停止。

右侧方法：自动收敛测试 (ϵ -Threshold Convergence Test)

预先设定一个小常数在每次迭代后，计算前后两次损失之差如果 $\Delta J \leq \varepsilon$ ，则判定“损失已经不再下降”，直接停止训练，认为已收敛。

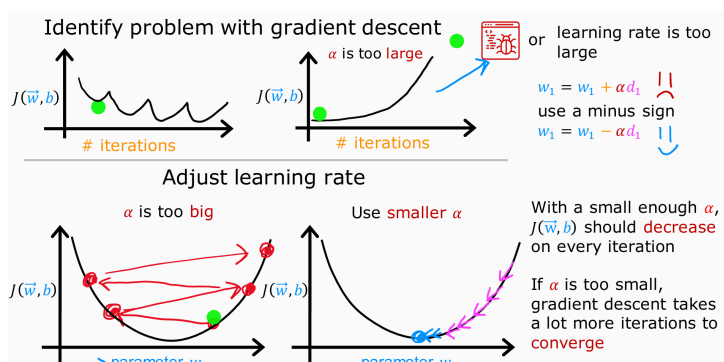
- **优点：**
 - 自动化程度高，一行代码即可实现；
 - 无需人工监控曲线，尤其适合大规模分布式训练或离线批训练场景。

• 缺点：

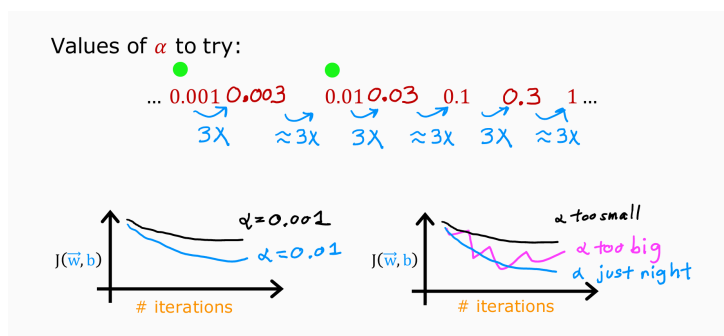
- ε 的选取需要经验：过大可能过早停下（未真正收敛），过小又可能浪费很多迭代；
- 仅关注单步下降幅度，对“震荡”或“指数尾部”收敛特性不够敏感。

特性	学习曲线可视化	ε -阈值自动收敛
直观性	很强；可看出整体收敛趋势与异常抖动	较弱；只给出是否“停止”二值结果
自动化程度	低；需要实时绘图 & 人工判断	高；完全可自动停止
阈值敏感性	通过图形“目测”灵活，不依赖固定阈值	依赖 ε 设计，对结果影响大
调参指导	能同时发现学习率过大/过小的问题	只能告诉你“何时停”，不指示方向问题
实现复杂度	中等；需搭建日志/可视化系统	低；在训练循环中简单比较即可
适用场景	调试 & 小规模实验	大规模生产 & 自动化训练

2.4 学习率的设置

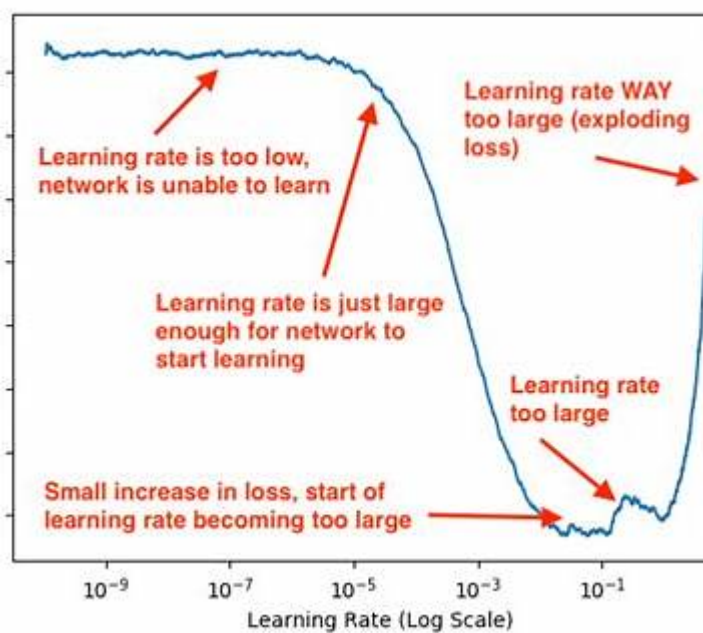
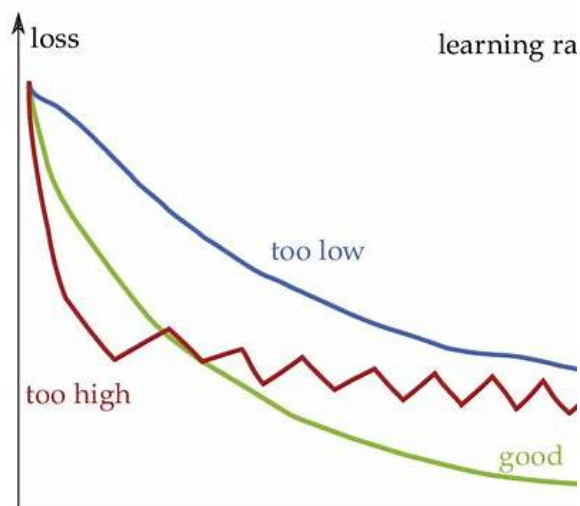
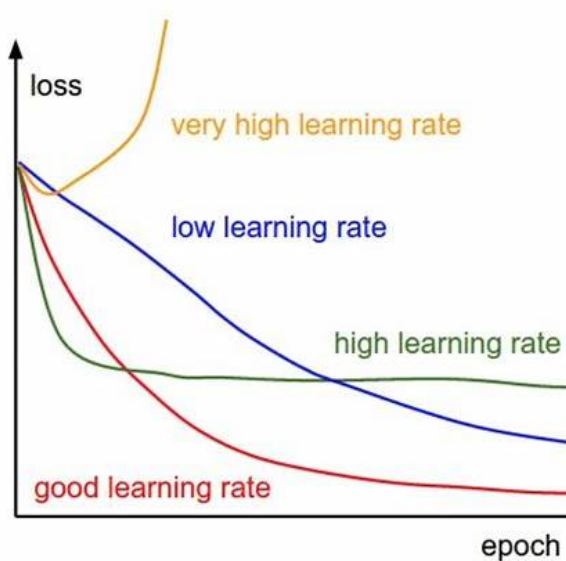


注意右上角的曲线，这个既可以代表学习率太大了，也可以代表着代码出现了bug（比如把梯度下降变成加法等等）



通常学习率的设置是按照最小的开始设置的，比如0.001。因为如果最小的学习率学习曲线还是不下降，那么就一定代表着代码出错了。

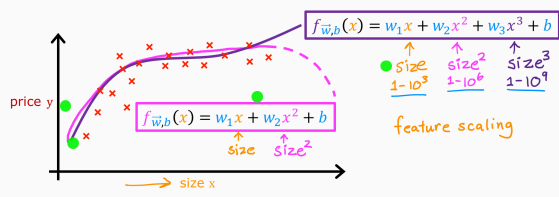
然后再逐渐调整学习率，直到学习曲线合理。



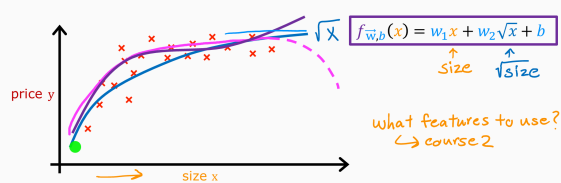
3.5 多项式回归

回归分析可以是多项式

Polynomial regression



Choice of features



逻辑回归

1. 逻辑回归

1.1 目的

逻辑回归是用来进行二分类的

Classification

Question

Answer "y"

Is this email spam?

no yes

Is the transaction fraudulent?

no yes

Is the tumor malignant?

no yes

y can only be one of two values

"binary classification"

class = category

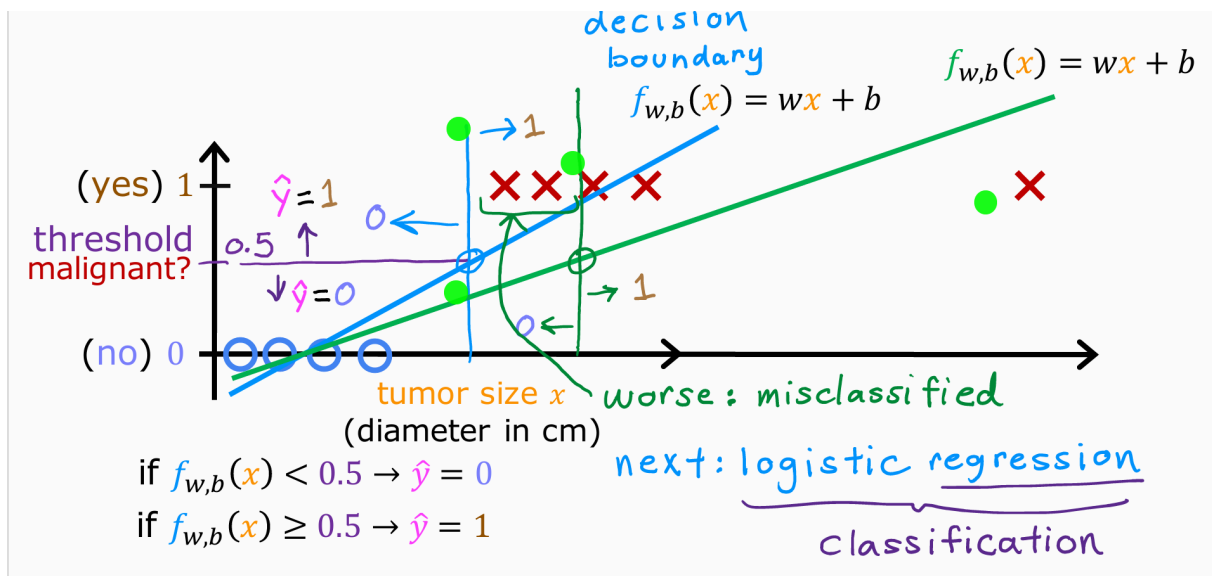
false true

0 1

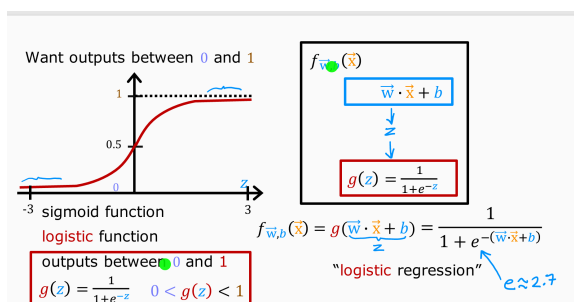
useful for classification

"negative class"
≠ "bad"
absence

"positive class"
≠ "good"
presence



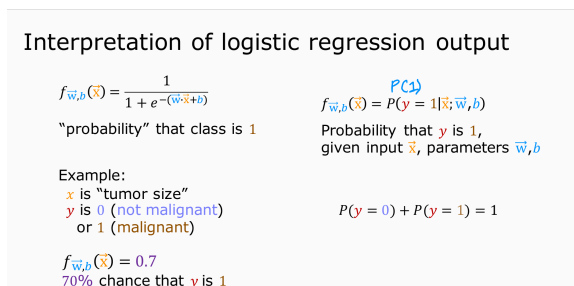
1.2 sigmoid函数



我们的目的是想要把回归步骤得到的数据变成一个概率，所以需要定义一个新的映射去把之前回归分析得到的y值映射到【0,1】区间内，于是就出现了sigmoid函数

这里经过sigmoid函数输出后的值落在了【0,1】之间，符合概率的定义

由于逻辑回归的y值只有0,1，所以逻辑回归得到的值抽象成是y是1的概率



1.3 决策边界

Decision boundary

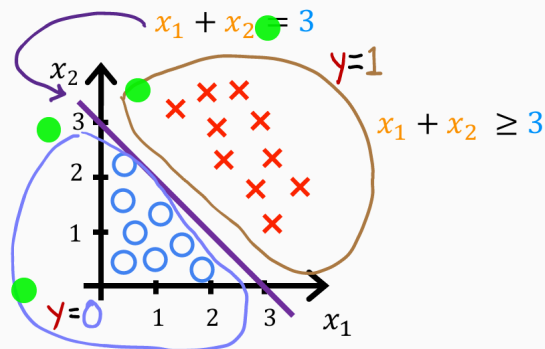
$$f_{\vec{w},b}(\vec{x}) = g(z) = g(\underbrace{w_1x_1 + w_2x_2 + b}_{z})$$

Decision boundary $z = \vec{w} \cdot \vec{x} + b = 0$

$$z = x_1 + x_2 - 3 = 0$$

$$x_1 + x_2 = 3$$

$$x_1 + x_2 < 3$$



根据sigmoid函数特点很容易得出当 $z=0$ 时是边界， $z=0$ 又对应着 $wx+b=0$ 。

同理也能知道当回归值 >0 时 y 是1，回归值 <0 时 y 是0

$$\text{Is } f_{\vec{w},b}(\vec{x}) \geq 0.5?$$

$$\text{Yes: } \hat{y} = 1$$

$$\text{No: } \hat{y} = 0$$

When is

$$f_{\vec{w},b}(\vec{x}) \geq 0.5 \quad g(z) \geq 0.5$$

$$z \geq 0$$

$$z < 0$$

$$\vec{w} \cdot \vec{x} + b \geq 0$$

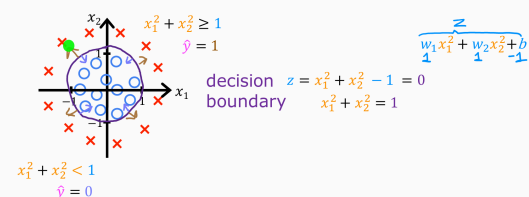
$$\vec{w} \cdot \vec{x} + b < 0$$

$$\hat{y} = 1$$

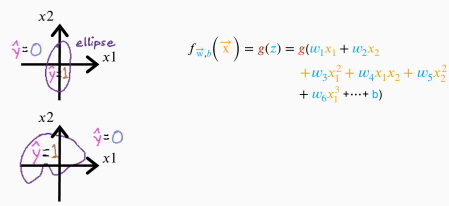
$$\hat{y} = 0$$

回归函数可以是多项式，所以也会出现接下来的情况：

Non-linear decision boundaries



Non-linear decision boundaries



2.逻辑回归的代价函数和损失函数

2.1 推导原理

从概率模型出发

对于一个样本 $\mathbf{x}$, 逻辑回归假设

$$P(y = 1 | \mathbf{x}) = h_{\theta}(\mathbf{x}), \quad P(y = 0 | \mathbf{x}) = 1 - h_{\theta}(\mathbf{x}),$$

其中 $h_{\theta}(\mathbf{x}) = \sigma(\theta^{\top} \mathbf{x})$ 。

那么这一个样本给定标签 y 的“似然”是

$$P(y | \mathbf{x}) = \begin{cases} h_{\theta}(\mathbf{x}), & y = 1, \\ 1 - h_{\theta}(\mathbf{x}), & y = 0. \end{cases}$$

我们要**最大化**所有样本的联合似然, 等价于**最小化**它的负对数似然。

对单个样本取负对数, 就得到

$$\text{Cost}(h_{\theta}(\mathbf{x}), y) = -\ln P(y | \mathbf{x}) = \begin{cases} -\ln(h_{\theta}(\mathbf{x})), & y = 1, \\ -\ln(1 - h_{\theta}(\mathbf{x})), & y = 0. \end{cases}$$

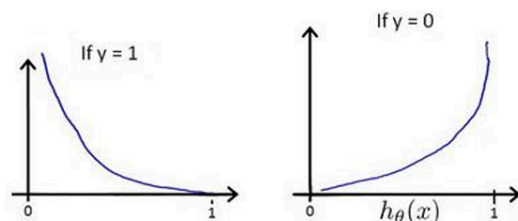
2.2 直观解释

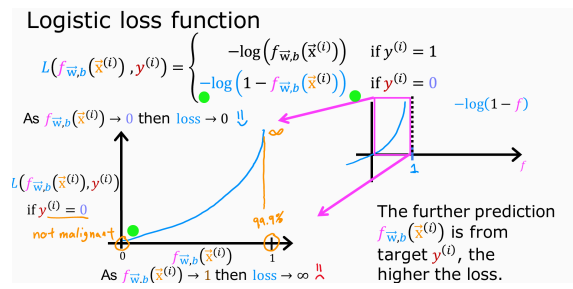
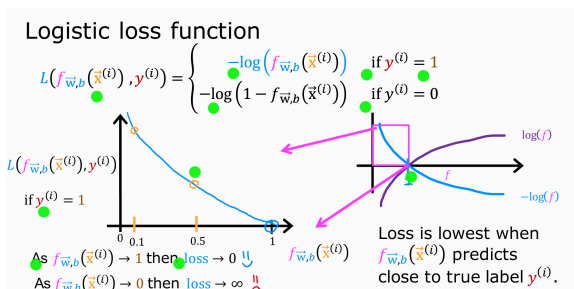
线性回归的代价函数为: $J(\theta) = \frac{1}{m} \sum_{i=1}^m \frac{1}{2} (h_{\theta}(x^{(i)}) - y^{(i)})^2$ 。

我们重新定义逻辑回归的代价函数为: $J(\theta) = \frac{1}{m} \sum_{i=1}^m \text{Cost}(h_{\theta}(x^{(i)}), y^{(i)})$, 其中

$$\text{Cost}(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

$h_{\theta}(x)$ 与 $\text{Cost}(h_{\theta}(x), y)$ 之间的关系如下图所示:





注意：y的取值只能是0-1，所以0-1的对数图像是我们研究的

左图是y=1时候的函数图像，可以看出当y=1（事情概率为真）的时候，如果f(x)=1那么就是对的，loss=0

右图同理。

2.3简化的代价函数

Simplified loss function

$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = \begin{cases} -\log(f_{\vec{w},b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 1 \\ -\log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = -y^{(i)}\log(f_{\vec{w},b}(\vec{x}^{(i)})) - (1 - y^{(i)})\log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))$$

if $y^{(i)} = 1$:

$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = -1 \log(f(\hat{x}))$$

if $y^{(i)} = 0$:

$$L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) = -(1-0)\log(1-f(\hat{x}))$$

Simplified cost function

$$\begin{aligned}
 \text{loss} \quad L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)}) &= -y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) - (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)})) \\
 \text{cost} \quad J(\vec{w}, b) &= \frac{1}{m} \sum_{i=1}^m [L(f_{\vec{w},b}(\vec{x}^{(i)}), y^{(i)})] \\
 &= -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))]
 \end{aligned}$$

convex
(single global minimum)

maximum likelihood
(don't worry about it!)

2.4 逻辑回归的梯度下降算法

Gradient descent

$$\begin{aligned}
 \text{cost} \quad J(\vec{w}, b) &= -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(f_{\vec{w},b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w},b}(\vec{x}^{(i)}))] \\
 \text{repeat } \{ & \\
 \quad j=1 \dots n & \quad w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b) \quad \frac{\partial}{\partial w_j} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \\
 \quad b &= b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b) \quad \frac{\partial}{\partial b} J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) \\
 \} & \text{ simultaneous updates}
 \end{aligned}$$

Gradient descent for logistic regression

repeat {

looks like linear regression!

$$w_j = w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right]$$

$$b = b - \alpha \left[\frac{1}{m} \sum_{i=1}^m (f_{\vec{w},b}(\vec{x}^{(i)}) - y^{(i)}) \right]$$

} simultaneous updates

Same concepts:

- Monitor gradient descent (learning curve)
- Vectorized implementation
- Feature scaling

Linear regression $f_{\vec{w},b}(\vec{x}) = \vec{w} \cdot \vec{x} + b$

Logistic regression $f_{\vec{w},b}(\vec{x}) = \frac{1}{1 + e^{(-\vec{w} \cdot \vec{x} + b)}}$

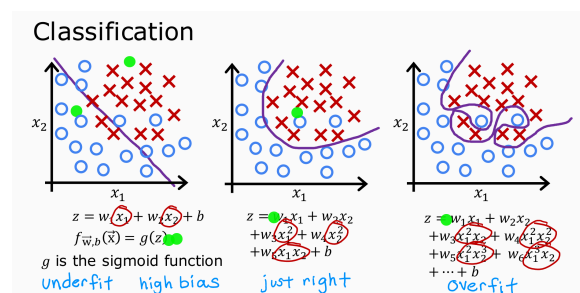
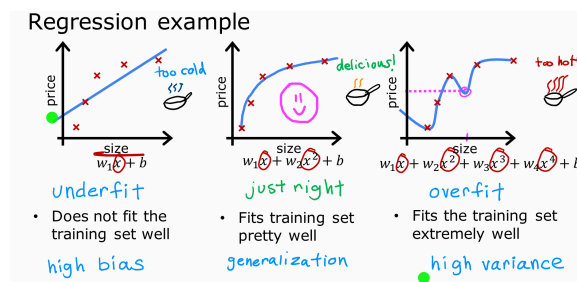
发现梯度下降求导后和回归问题的梯度下降算法一样了，都是(预测-标签)×特征。

这是选择sigmoid函数的另一个原因

过拟合与正则化

1.过拟合

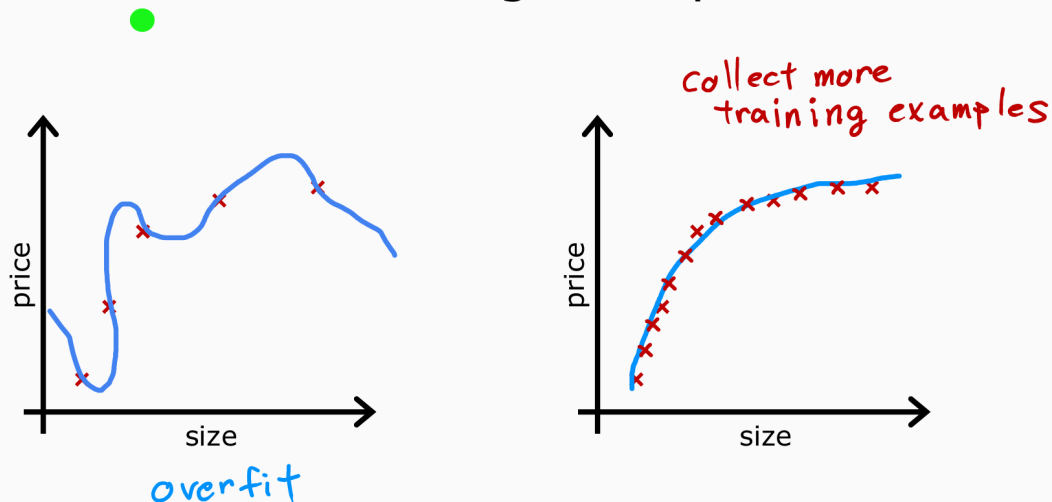
1.1 直观理解过拟合和欠拟合



1.2 解决方法

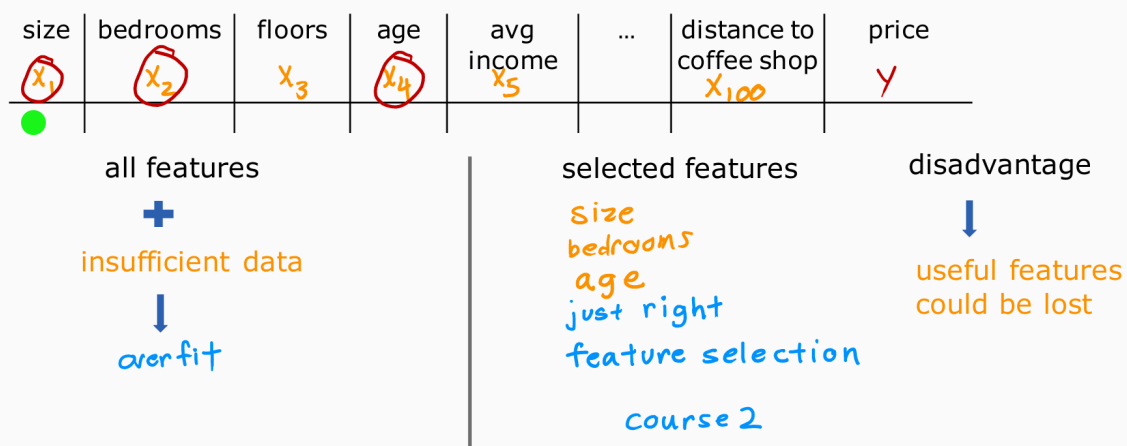
方法一：加入很多的训练样本

Collect more training examples



方法二：选择重要的特征向量，不重要的舍去

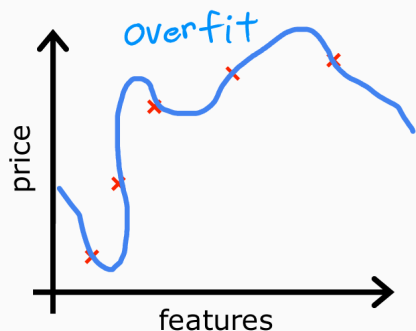
Select features to include/exclude



方法三：想办法让影响因子大的特征值变小（正则化）

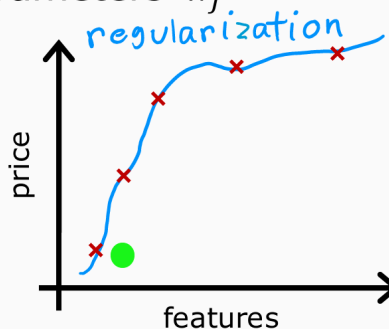
Regularization

Reduce the size of parameters w_j



$$f(x) = 28x - 385x^2 + 39x^3 - 174x^4 + 100$$

large values for w_j → eliminate feature



$$f(x) = 13x - 0.23x^2 + 0.000014x^3 - 0.0001x^4 + 10$$

small values for w_j

2. 正则化

2.1 数学形式

上面的方法三其实就是对 w_3, w_4 进行正则化。

但是实际上我们不知道一个模型中哪个参数才是重要的，刚刚的例子是我们知道了其实 w_3, w_4 是不重要的才这么做，所以正则化应该是对所有参数进行的。

所以正则化的形式应该像下面图这样：期中由于 b 是噪声（bias）所以正不正则化都无所谓

Regularization

small values $w_1, w_2, \dots, w_n, b$

simpler model

less likely to overfit

$w_3 \approx 0$

$w_4 \approx 0$

size x_1	bedrooms x_2	floors x_3	age x_4	avg income x_5	...	distance to coffee shop x_{100}	price y
$w_1, w_1, w_2, \dots, w_{100}, b$							
n features							
n = 100							

$$J(\vec{w}, b) = \frac{1}{2m} \left[\sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})^2 + \underbrace{\frac{\lambda}{2m} \sum_{j=1}^n w_j^2}_{\text{"lambda" regularization parameter}} + \underbrace{\frac{\lambda}{2m} b^2}_{\text{regularization term}} \right]$$

can include or exclude b

$\lambda > 0$

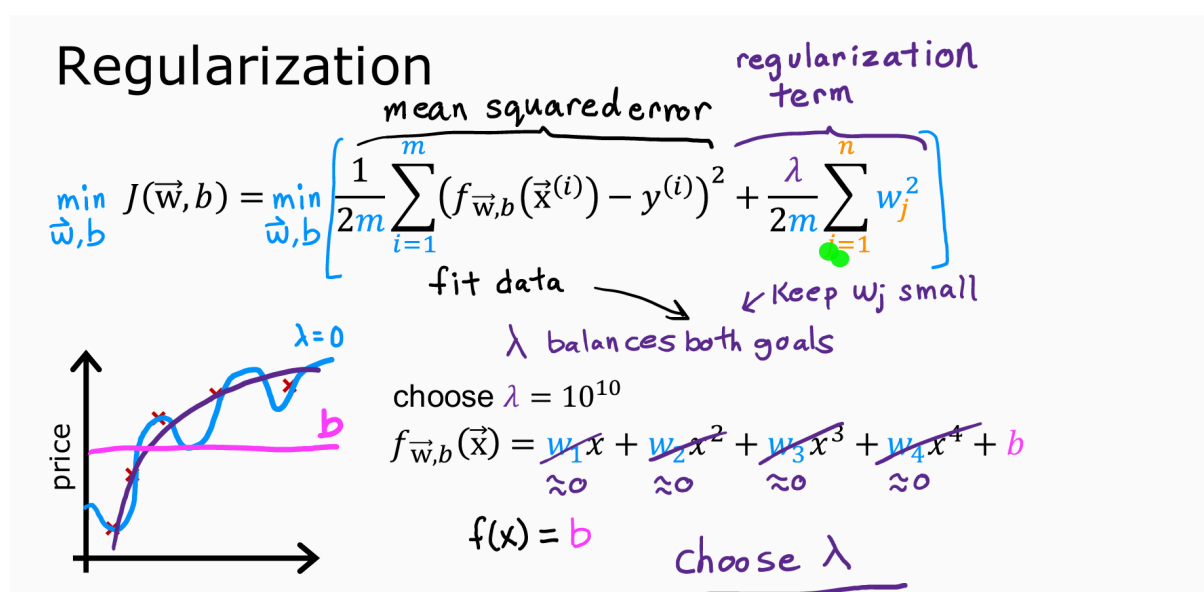
引入 $2m$ 同样是为了求导方便

2.2 正则化参数 λ

把正则化后的代价函数写出来，不难看出 λ 的值其实决定了正则化的强度。

λ 太小了则会导致正则化没什么用，还是原来的代价函数。

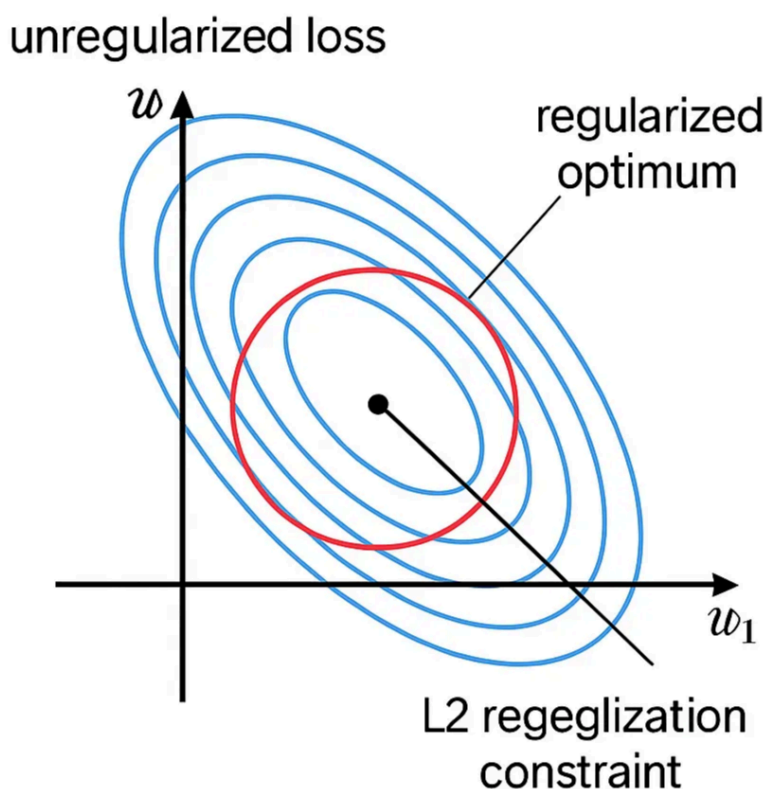
λ 太大了会导致梯度下降着重于后面的正则化表达式，从而会强调求 w 的最小值，最后会导致所有 w 都趋近于0



- **交叉验证 (Cross-Validation):** 这是最常用且比较可靠的方法。
 - **k-折交叉验证 (k-Fold Cross-Validation):**
 1. 将原始训练数据集随机分成 k 个互斥的子集（称为“折”）。
 2. 选取一组候选的 λ 值（例如，对数尺度上的一系列值：0.001, 0.01, 0.1, 1, 10, 100）。
 3. 对于每一个候选 λ 值：
 - 进行 k 次训练和验证：每次选择 $k-1$ 折作为训练集，剩下的 1 折作为验证集。
 - 训练模型，并在验证集上评估性能（例如，计算准确率、均方误差等）。
 - 记录 k 次验证结果的平均性能。
 4. 比较所有候选 λ 值对应的平均验证性能，选择那个使性能最优的 λ 值作为最终选定的超参数。

- **留出验证 (Hold-out Validation):** 简单做法是将训练数据再分出一部分作为固定的验证集。尝试不同的 λ 值，在训练集上训练模型，然后在验证集上评估。选择在验证集上表现最好的 λ 。这种方法不如 k-折交叉验证稳健，尤其是在数据量不大的情况下。

2.3 原理解释



1. 无正则化的等高线 (蓝色椭圆)

- 每条蓝色曲线代表在权重空间 (w_1, w_2) 上，损失函数取相同值的点集合。
- 椭圆形状源自不同方向梯度与特征尺度的差异：若 w_1 对损失的影响比 w_2 大，则沿 w_1 方向收缩更快，形成偏长的椭圆。

2. L2 正则化约束 (红色圆)

- L2 惩罚项 $\frac{\lambda}{2m} \sum_j w_j^2$ 对应于在权重空间中限制 $\|\mathbf{w}\|^2$ 不超过某个常数的圆形区域。
- 这条红色圆表示所有满足 $\sum w_j^2 = C$ 的点。

3. 正则化后最优解 (黑点)

- 原始最优解位于最内层蓝色椭圆的中心，但可能不在红色圆内。
- 添加约束后，新的最优解必须同时位于红色圆和某条蓝色椭圆上，故在它们切点处：
 - 它是距离原点最近且损失值相同的点， $\|\mathbf{w}\|$ 被强制更小。
 - 这样权重整体缩小，减弱了对训练噪声的拟合能力，降低了模型复杂度，缓解过拟合。

2.4 回归问题的正则化

先给出推导，注意没有sigma符号。

原因是求导过程中只对当前的j起作用，其余的所有w都无效，是0。

How we get the derivative term (optional)

$$\begin{aligned}
 \frac{\partial}{\partial w_j} J(\vec{w}, b) &= \frac{\partial}{\partial w_j} \left[\frac{1}{2m} \sum_{i=1}^m \left(\underbrace{f(\vec{x}^{(i)})}_{\vec{w} \cdot \vec{x}^{(i)} + b} - y^{(i)} \right)^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2 \right] \\
 &= \frac{1}{2m} \sum_{i=1}^m \left[\left(\vec{w} \cdot \vec{x}^{(i)} + b - y^{(i)} \right) \cancel{2x_j^{(i)}} \right] + \frac{\lambda}{2m} \cancel{2w_j} \quad \text{No } \sum_{j=1}^n \\
 &= \frac{1}{m} \sum_{i=1}^m \left[\underbrace{(\vec{w} \cdot \vec{x}^{(i)} + b - y^{(i)})}_{f(\vec{x})} x_j^{(i)} \right] + \frac{\lambda}{m} w_j \\
 &= \frac{1}{m} \sum_{i=1}^m \left[(f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right] + \frac{\lambda}{m} w_j
 \end{aligned}$$

Implementing gradient descent

repeat {

$$w_j = w_j - \alpha \left[\frac{1}{m} \sum_{i=1}^m \left[(f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right] + \frac{\lambda}{m} w_j \right]$$

$$b = b - \alpha \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})$$

} simultaneous update $j = 1 \dots n$

$$w_j = \underbrace{w_j - \alpha \frac{\lambda}{m} w_j}_{w_j \left(1 - \alpha \frac{\lambda}{m} \right)} - \underbrace{\alpha \frac{1}{m} \sum_{i=1}^m (f_{w, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)}}_{\text{usual update}}$$

shrink w_j

$$\begin{aligned}
 &\alpha \frac{\lambda}{m} \\
 &0.01 \frac{1}{50} = 0.0002 \\
 &\frac{w_j (1 - 0.0002)}{0.9998}
 \end{aligned}$$

从上面的推导中也能看出来lamda的影响，lamda会让每一次梯度下降更新都让wj减小一点点。

2.5 逻辑回归中的正则化

Regularized logistic regression

$$\min_{\vec{w}, b} J(\vec{w}, b) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(f_{\vec{w}, b}(\vec{x}^{(i)})) + (1 - y^{(i)}) \log(1 - f_{\vec{w}, b}(\vec{x}^{(i)})) \right] + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

Gradient descent

repeat {

$$w_j = w_j - \alpha \frac{\partial}{\partial w_j} J(\vec{w}, b)$$

$j = 1 \dots n$

$$b = b - \alpha \frac{\partial}{\partial b} J(\vec{w}, b)$$

}

Looks same as
for linear regression!

$$= \frac{1}{m} \sum_{i=1}^m \left[(f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)}) x_j^{(i)} \right] + \frac{\lambda}{m} w_j$$

logistic regression

$$= \frac{1}{m} \sum_{i=1}^m (f_{\vec{w}, b}(\vec{x}^{(i)}) - y^{(i)})$$

don't have to
regularize b

决策树

1. 决策树模型

决策树是一种基础的分类与回归方法。它通过一系列“问题”或“条件”来对数据进行划分，最终达到决策的目的。其结构像一棵倒置的树。

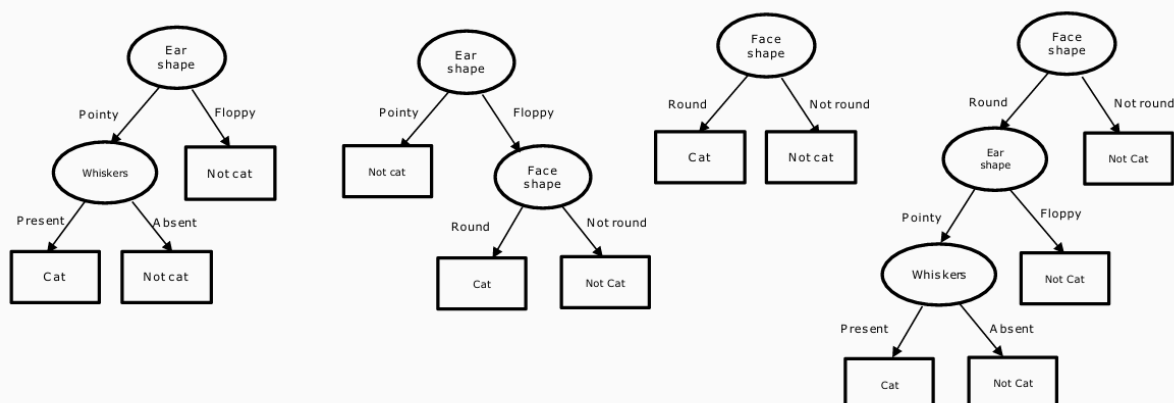
- 核心组成部分

- **根节点 (Root Node):** 整棵树的起点，包含了所有的训练样本。
- **决策节点 (Decision Nodes):** 也叫内部节点，代表一个特征或属性的测试。每个节点会根据测试结果将数据分裂成两个或多个子节点。
- **分支 (Branches):** 代表决策节点上每个可能的测试结果。
- **叶节点 (Leaf Nodes):** 树的终点，代表一个最终的决策结果（如分类的类别或回归的数值）。

- 工作原理

决策树通过从根节点开始，根据新样本的特征值，沿着决策路径一步步向下，最终到达一个叶节点，并将该叶节点的类别或数值作为预测结果 2。

Decision Tree



2.决策树学习过程

构建决策树的核心在于如何选择最优的特征进行分裂，以及何时停止分裂。这个过程通常是一个递归的、贪心的算法。

2.1. 如何选择分裂特征

目标是在每个节点上找到一个特征，使得分裂后的数据“纯度”最高（或“不纯度”最低）。

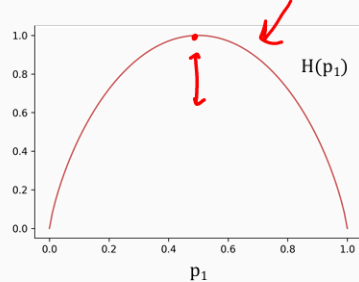
- 纯度/不纯度的度量

一个节点中样本类别越一致，纯度越高，不纯度越低。

- **熵 (Entropy):** 物理学中表示系统的混乱程度，在信息论中用来度量信息的不确定性。熵越大，不纯度越高，不确定性越大。
 - 公式为： $H(p1) = -p1\log_2(p1) - p0\log_2(p0)$ ，其中 $p1$ 是类别1（如猫）的比例， $p0$ 是类别0的比例 ($p0=1-p1$)。

Entropy as a measure of impurity

p_1 = fraction of examples that are cats



$$p_0 = 1 - p_1$$

$$H(p_1) = -p_1 \log_2(p_1) - p_0 \log_2(p_0)$$

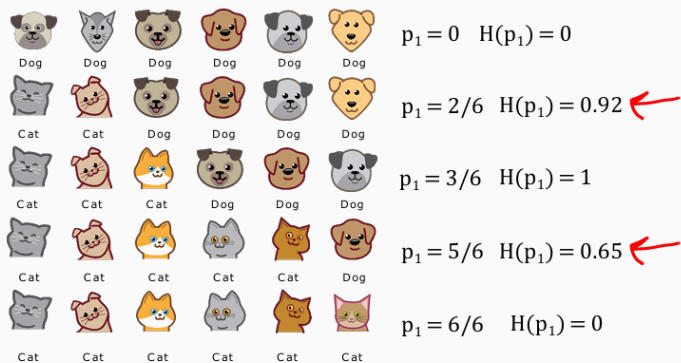
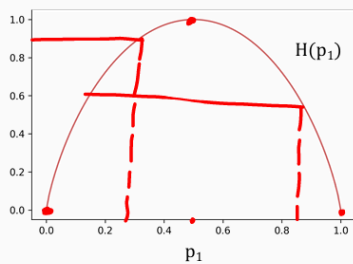
$$= -p_1 \log_2(p_1) - (1 - p_1) \log_2(1 - p_1)$$

Note: " $0 \log(0)$ " = 0

- 当节点完全纯净时（全是猫或全不是猫）， $p_1=1$ 或 $p_1=0$ ，熵 $H=0$ 。
- 当节点最混乱时（一半是猫，一半不是）， $p_1=0.5$ ，熵 $H=1$ 达到最大值。

Entropy as a measure of impurity

p_1 = fraction of examples that are cats



- **信息增益 (Information Gain):** 表示通过某个特征进行分裂后，系统不确定性（熵）减少的程度。信息增益越大，说明使用该特征进行分裂的效果越好。

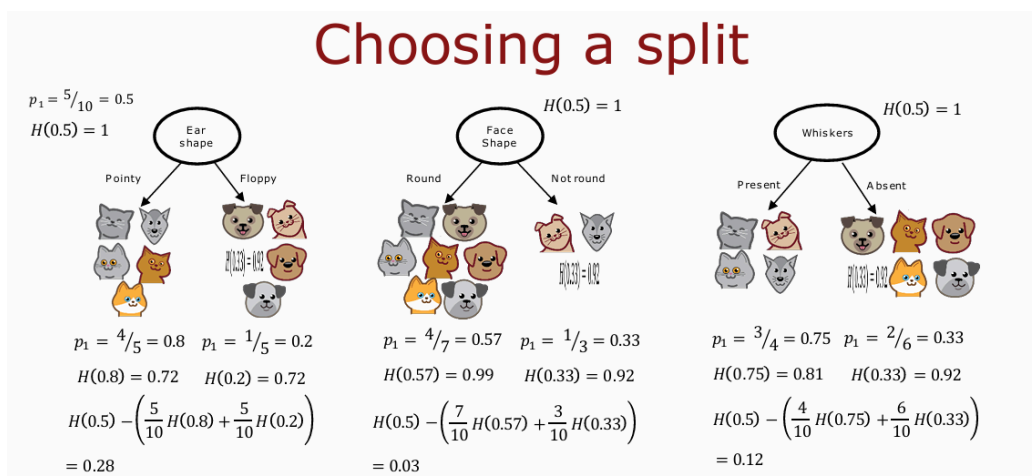
- 公式:

Information gain

$$= H(p_1^{\text{root}}) - \left(w^{\text{left}} H(p_1^{\text{left}}) + w^{\text{right}} H(p_1^{\text{right}}) \right)$$

其中， $H(p_1^{\text{root}})$ 是父节点的熵， $H(p_1^{\text{left}})$ 和 $H(p_1^{\text{right}})$ 分别是左右子节

点的熵，wleft 和 wright 是左右子节点样本数占父节点样本数的权重。



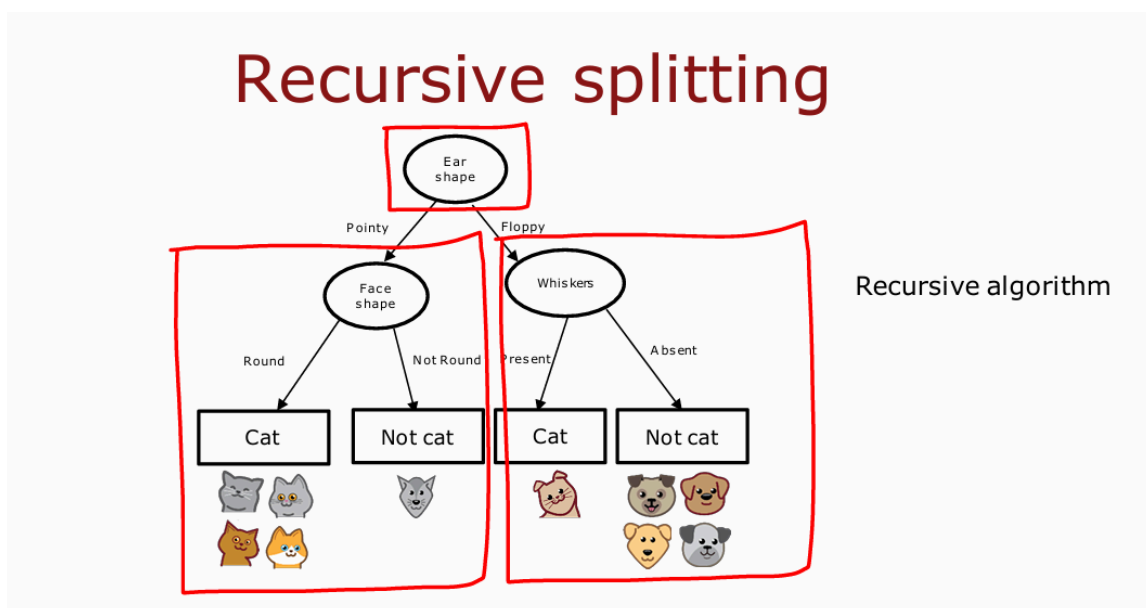
- 学习算法会在所有可能的特征中，选择**信息增益最大（熵减少的最大）**的那个特征作为当前节点的分裂标准。
- **扩展：其他纯度度量标准**
 - **基尼不纯度 (Gini Impurity):** CART (Classification and Regression Tree) 算法使用的标准。它度量了从数据集中随机选择一个样本，然后根据该数据集中样本的分布情况，将其错误分类的概率。
 - 公式: $Gini = 1 - \sum_{i=1}^k p_i^2$ ，其中 p_i 是类别 i 的比例。
 - 基尼不纯度的计算比熵更快（因为它没有对数运算），因此在很多库（如scikit-learn）中是默认选项。其趋势与熵类似，纯净节点 $Gini=0$ ，最混乱时 $Gini=0.5$ （对于二分类）。
 - **信息增益率 (Information Gain Ratio):** C4.5算法使用。信息增益倾向于选择那些具有大量取值的特征（例如ID号码），因为每个值都能形成一个分支，使得子节点纯度极高。信息增益率通过引入一个惩罚项（分裂信息度量，Intrinsic Value）来校正这种偏见。

2.2 何时停止分裂 (When to Stop Splitting)

如果无限制地生长，决策树会变得异常复杂，导致**过拟合 (Overfitting)**，即在训练数据上表现完美，但在新数据上表现很差。因此需要设定停止条件，这被称为**预剪枝 (Pre-pruning)**。

- **常见的停止条件:**
 1. 当一个节点中的所有样本都属于同一类别时（即节点100%纯净）。
 2. 当树的深度达到预设的**最大深度 (maximum depth)** 时。

3. 当分裂带来的信息增益小于某个阈值时。
4. 当节点中的样本数量少于某个阈值时。



- 扩展：后剪枝 (Post-pruning)

与预剪枝（在生长过程中提前停止）相对，后剪枝是先让决策树完全生长，然后再自底向上地考察各个节点。如果剪掉（合并）一个分支能提升模型在验证集上的性能，就执行剪枝。后剪枝通常比预剪枝能保留更多有用的分支，模型性能可能更好，但计算开销也更大。

3 处理不同类型的特征

3.1. 分类特征与独热编码

当一个分类特征有超过两个以上的取值时（例如，“耳朵形状”可能有“尖的”、“塌的”、“椭圆的”三种），直接使用这个特征可能会给模型带来困惑。

解决方案：独热编码 (One-Hot Encoding)

独热编码是一种将多分类型特征转换为多个二元（0或1）特征的技术。

- **如何操作：**如果有一个叫“耳朵形状”的特征有3个值（尖、塌、椭圆），我们就创建3个新的二元特征：“是尖耳朵吗？”、“是塌耳朵吗？”、“是椭圆耳朵吗？”。
 - 对于一个“尖耳朵”的样本，它在“是尖耳朵吗？”这个新特征上的值为1，在另外两个特征上的值为0。

One hot encoding

	Ear shape	Pointy ears	Floppy ears	Oval ears	Face shape	Whiskers	Cat
	Pointy	1	0	0	Round	Present	1
	Oval	0	0	1	Not round	Present	1
	Oval	0	0	1	Round	Absent	0
	Pointy	1	0	0	Not round	Present	0
	Oval	0	0	1	Round	Present	1
	Pointy	1	0	0	Round	Absent	1
	Floppy	0	1	0	Not round	Absent	0
	Oval	0	0	1	Round	Absent	1
	Floppy	0	1	0	Round	Absent	0
	Floppy	0	1	0	Round	Absent	0

扩展知识：独热编码的意义

- **消除序数关系**：对于决策树来说，如果将“尖、塌、椭圆”编码为1, 2, 3，模型可能会错误地认为它们之间存在某种顺序。独热编码将它们变成了相互独立、地位平等的特征，消除了这种隐含的、不正确的顺序假设。
- **广泛应用**：这种编码方式不仅用于决策树，在神经网络等许多机器学习模型中也是处理分类数据的标准做法。

3.2 连续值特征

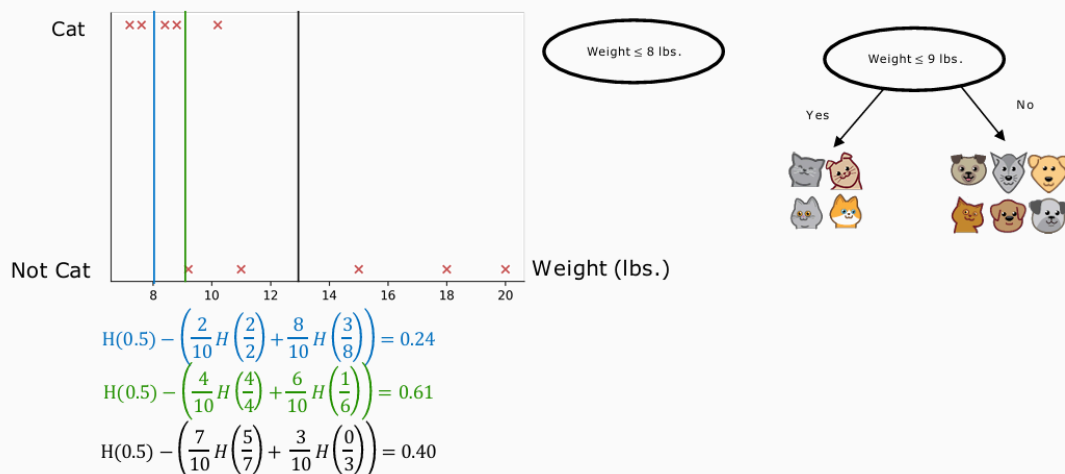
当特征是连续的数值时，比如体重，我们无法像分类特征那样穷举所有的值进行拆分。

解决方案：寻找最佳分割点

算法会将所有样本按该连续特征的值进行排序，然后尝试每一个可能的分割点，计算其信息增益。

- **如何操作**：算法会遍历所有可能的分割点（例如，`体重 <= 7.4`，`体重 <= 8.0`，...），最终选择那个能带来**最大信息增益**的分割点作为拆分规则。

Splitting on a continuous variable



4. 回归树 (Regression Trees)

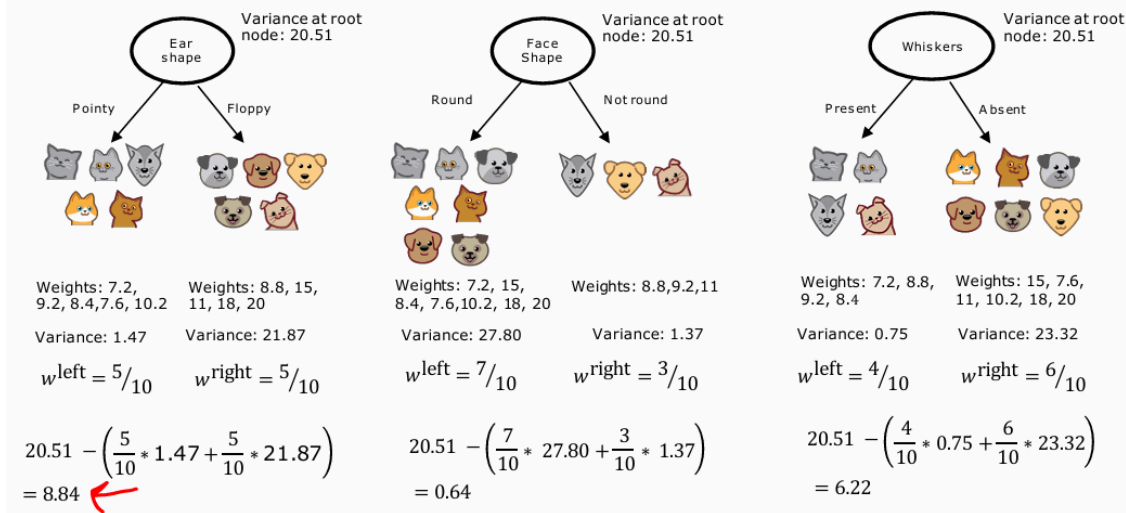
决策树不仅能用于分类，也能用于回归，即预测一个连续值（如房价、体重）。

Regression with Decision Trees: Predicting a number

	Ear shape	Face shape	Whiskers	Weight (lbs.)
	Pointy	Round	Present	7.2
	Floppy	Not round	Present	8.8
	Floppy	Round	Absent	15
	Pointy	Not round	Present	9.2
	Pointy	Round	Present	8.4
	Pointy	Round	Absent	7.6
	Floppy	Not round	Absent	11
	Pointy	Round	Absent	10.2
	Floppy	Round	Absent	18
	Floppy	Round	Absent	20

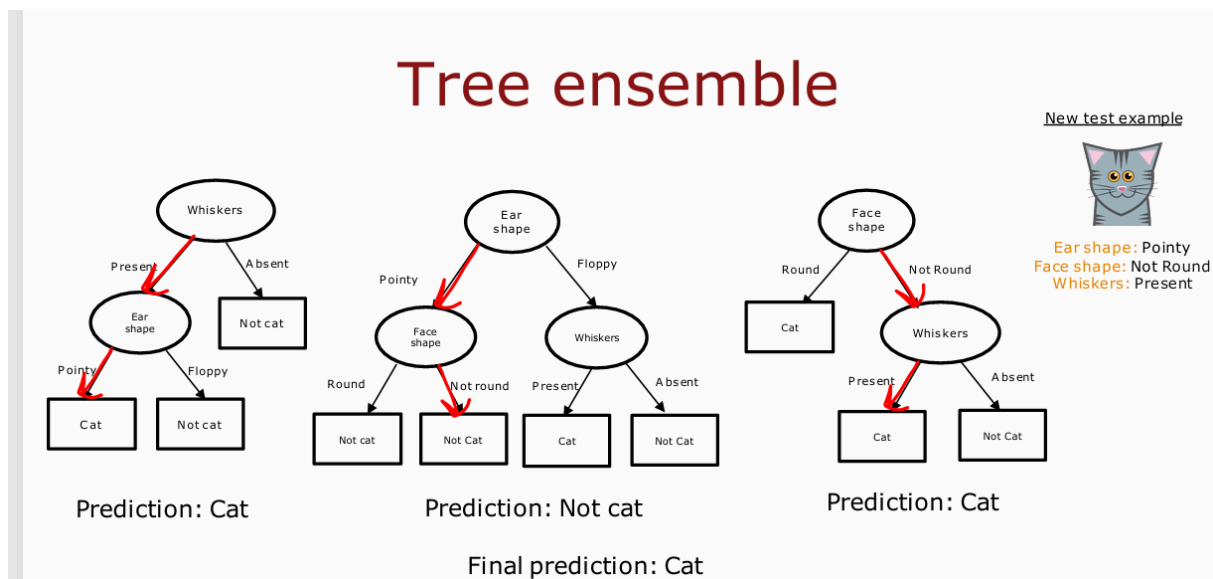
- **分裂标准:** 回归树不再使用熵或基尼不纯度。它使用**方差 (Variance)** 或**均方误差 (Mean Squared Error, MSE)** 作为度量标准。目标是选择一个分裂，使得分裂后的两个子节点内的样本目标值方差之和最小。
 - **方差减小量:** 类似于信息增益，算法会选择使方差减小量最大的特征和阈值进行分裂。计算方式为： $\text{Variance}_{\text{parent}} - (\text{w}_{\text{left}} \text{Variance}_{\text{left}} + \text{w}_{\text{right}} \text{Variance}_{\text{right}})$ 。
- **预测值:** 对于一个回归树，叶节点的预测值是该节点内所有训练样本目标值的**平均值**。

Choosing a split



5. 树集成 (Tree Ensembles)

单个决策树非常不稳定，训练数据的微小变化可能导致生成一棵完全不同的树，这被称为**高方差 (High Variance)**。树集成方法通过构建并结合多个决策树来获得更稳定、更准确的预测。



5.1. 随机森林 (Random Forest)

随机森林是**Bagging (Bootstrap Aggregating)** 的一种扩展，它通过引入双重随机性来构建一组各不相同的决策树。

- **核心理念:**

1. **数据随机性 (Bagging):** 通过有放回抽样 (Sampling with replacement) 的方式, 从原始 m 个训练样本中抽取 m 个样本, 形成一个新的训练集。这个过程重复 B 次, 得到 B 个不同的训练集。由于是有放回的, 每个新数据集中会包含重复样本, 且约有 $1/3$ 的原始数据未被抽中。
2. **特征随机性:** 在训练每一棵树时, 当需要在节点上选择分裂特征时, 并不是从所有 n 个特征中选择, 而是随机选取一个由 k 个特征组成的子集 ($k < n$), 然后从这个子集中选择最优分裂特征。一个常见的做法是 $k=n$ 。

- **预测:**

- **分类问题:** B 棵树进行投票, 得票最多的类别为最终结果。
- **回归问题:** 取 B 棵树预测值的平均值。

- **优点:** 极大地降低了模型的方差, 能有效防止过拟合, 性能强大且稳定。

5.2. 提升树 (Boosting) 与 XGBoost

- **基本思想 (Boosting):** 树是依次 (sequentially) 构建的, 每一棵新树都致力于纠正前面所有树犯下的错误。
 - PPT中的直观解释是: 在为下一棵树构建训练集时, 会提高那些被前面树错误分类的样本的权重 (抽样概率), 使得新树更加关注这些“难啃的”样本。

Boosted trees intuition

Given training set of size m

For $b = 1$ to B :

Use sampling with replacement to create a new training set of size m
 But instead of picking from all examples with equal ($1/m$) probability, make it more likely to pick examples that the previously trained trees misclassify

Train a decision tree on the new dataset

Ear shape	Face shape	Whiskers	Cat
Pointy	Round	Present	Yes
Floppy	Round	Absent	No
Round	Round	Absent	No
Pointy	Round	Present	Yes
Pointy	Not Round	Present	Yes
Floppy	Round	Absent	No
Floppy	Round	Present	Yes
Pointy	Not Round	Absent	No
Pointy	Not Round	Absent	No
Pointy	Not Round	Present	Yes

Ear shape	Face shape	Whiskers	Prediction
Pointy	Round	Present	Cat
Floppy	Round	Present	Not cat
Floppy	Round	Absent	Not cat
Pointy	Not Round	Present	Not cat
Pointy	Round	Present	Cat
Round	Round	Absent	Not cat
Pointy	Not Round	Absent	Not cat
Floppy	Round	Absent	Not cat
Pointy	Not Round	Absent	Not cat
Floppy	Round	Absent	Not cat

$1, 2, \dots, b-1$ (pointing to previous trees) and b (pointing to current tree)

- **XGBoost (eXtreme Gradient Boosting):**
 - 它是**梯度提升 (Gradient Boosting)** 算法的一种高效、灵活的工程实现。
 - **XGBoost的优势:**
 - **速度快、效率高:** 对算法进行了并行化等优化。

- **内置正则化:** 在目标函数中加入了L1和L2正则化项，可以有效防止过拟合。
- **默认参数优秀:** 提供了很好的默认参数，开箱即用。
- **高度的灵活性和可扩展性。**
- XGBoost 在数据科学竞赛（如Kaggle）中极为流行且表现优异。它可以同时用于分类（`XGBClassifier`）和回归（`XGBRegressor`）。
- **扩展知识：其他Boosting框架:**
 - **LightGBM:** 微软出品，速度比XGBoost更快，内存占用更低，尤其擅长处理大规模数据。
 - **CatBoost:** Yandex出品，对分类特征有非常出色的原生支持，无需手动进行独热编码，通常能取得很好的效果。

6. 决策树 vs. 神经网络

特性	决策树 & 树集成	神经网络
适用数据	在表格数据/结构化数据上表现极佳。	适用于所有类型的数据，包括非结构化数据（图像、音频、文本）和结构化数据。
性能	训练和预测速度通常非常快。	训练可能很慢，尤其对于大型网络和数据集。
可解释性	单个的小决策树可解释性强，但复杂的集成模型（如XGBoost）则像黑箱。	通常被认为是“黑箱模型”，可解释性较差。
特征工程	对特征缩放不敏感。	通常需要对数据进行归一化或标准化。
迁移学习	不支持。	支持迁移学习，可以利用预训练模型来处理新任务，在非结构化数据领域尤其重要。

7. 总结：决策树的优缺点

- **优点:**
 1. **可解释性强:** 单个决策树的决策逻辑清晰，易于理解和可视化。
 2. **数据预处理要求低:** 不需要进行特征归一化或标准化。
 3. **能同时处理分类和连续特征。**

4. **非参数模型**: 对数据分布没有先验假设。

- **缺点**:

1. **容易过拟合**: 单个决策树倾向于学习训练数据中的噪声, 导致泛化能力差。

2. **不稳定性**: 训练数据的微小变化可能导致生成完全不同的树。

3. **贪心算法**: 决策树的构建是基于局部最优选择 (当前节点最好的分裂), 无法保证得到全局最优的树。

4. **类别不平衡问题**: 对于类别不平衡的数据集, 信息增益等标准会偏向于样本数较多的类别。

深度学习部分见李沐深度学习笔记

强化学习与推荐系统部分见对应笔记